

Documentación técnica de AeroReserva

Arquitectura de microservicios con Astro, Ruby on Rails, Cassandra, gRPC y Docker

Dato	Información
Grupo	Grupo 5
Integrantes	Samuel Daza, Samuel Velandia y Sara Muñoz
Profesor	Andrés Armando Sánchez Martín
Fecha	29/09/2026
Repositorio	https://github.com/sara-munoz-5/taller-microservicios
Versión o tag	[COMPLETAR AL FINAL]

Índice

Resumen ejecutivo	5
1. Investigación	5
1.1. Microservicios	5
1.1.1. Historia	6
1.1.2. Características	7
1.1.3. Problemas y desventajas	7
1.1.4. Patrones y cuándo usarlos	8
1.1.5. Ejemplos de la industria	9
1.2. Astro	10
1.2.1. Definición: qué es y qué no es	10
1.2.2. Características principales	10
1.2.3. Historia y evolución	10
1.2.4. Ventajas y desventajas	11
1.2.5. Casos de uso: cuándo usarlo y cuándo no	11
1.2.6. Casos de aplicación en la industria	11
1.2.7. Relación con el estilo de microservicios y las tecnologías seleccionadas	11
1.2.8. Frecuencia de uso del <i>stack</i> combinado	12
1.3. gRPC	12
1.3.1. Historia	13
1.3.2. Características	13
1.3.3. Ventajas y desventajas	14
1.3.4. Casos de uso	15
1.3.5. Casos en la industria	15
1.3.6. Relación entre el estilo y gRPC	15
1.3.7. Qué tan común es el <i>stack</i> designado	16
1.4. Ruby on Rails	16
1.4.1. Definición: qué son y qué no son	16
1.4.2. Características principales	16
1.4.3. Historia y evolución	16
1.4.4. Ventajas y desventajas	17
1.4.5. Casos de uso: cuándo usarlos y cuándo no	17
1.4.6. Casos de aplicación en la industria	17
1.4.7. Relación con el estilo de microservicios y las tecnologías seleccionadas	17
1.4.8. Frecuencia de uso del <i>stack</i> combinado	18

1.5.	Cassandra	18
1.5.1.	Definición clara: qué es y qué no es	18
1.5.2.	Características principales	18
1.5.3.	Historia y evolución	19
1.5.4.	Ventajas y desventajas	19
1.5.5.	Casos de uso: cuándo usarla y cuándo no	20
1.5.6.	Casos de aplicación real	20
1.6.	Comparación de alternativas y síntesis	20
2.	Contexto y alcance	22
2.1.	Alcance funcional	23
2.2.	Requisitos	23
2.3.	Atributos de calidad — ISO/IEC 25010:2023	23
3.	Arquitectura y justificación	24
3.1.	Componentes principales	25
3.2.	Diagramas C4	26
4.	Backend, gRPC y flujo de negocio	30
4.1.	Flujo de creación de reserva	32
4.2.	Endpoints BFF	33
5.	Cassandra y modelo de datos	34
5.1.	Espacio de datos y tablas de reservas	35
6.	Despliegue y operación	36
6.1.	Dockerfiles	36
6.2.	docker-compose.yml	37
6.3.	cassandra-init	38
6.4.	Healthchecks	38
6.5.	Ejecución del sistema	38
6.6.	Matriz de ejecución y verificación	39
6.7.	Riesgos operativos y de despliegue	39
6.8.	Guion de demostración	40
7.	Calidad, pruebas y resiliencia	41
7.1.	Atributos de calidad evaluados	41
7.2.	Pruebas de integración	42

7.3. Saga	43
7.4. Circuit Breaker	43
7.5. Control de cupos	44
7.6. Patrones de diseño GoF implementados	44
8. Decisiones arquitectónicas ADR	45
ADR-001. Separar Vuelos, Pasajeros y Reservas en microservicios	45
ADR-002. Usar gRPC y Protocol Buffers entre servicios	45
ADR-003. Usar Cassandra con un keyspace por servicio	46
ADR-004. Usar Astro como frontend y BFF	46
ADR-005. Manejo de fallos distribuidos (Saga y Circuit Breaker)	47
9. Limitaciones y trabajo futuro	48
9.1. Identificación en lugar de autenticación	48
9.2. Negocio fuera de alcance	49
9.3. Despliegue limitado a una sola máquina	49
9.4. Cassandra en una sola instancia física	49
9.5. Resiliencia no validada bajo carga real	49
9.6. Observabilidad básica	49
9.7. Trabajo futuro	50
10. Análisis arquitectónico	50
10.1. Matriz de atributos de calidad vs. estilo	50
10.2. Matriz de principios de diseño vs. estilo	51
10.3. Matriz de tácticas arquitectónicas vs. estilo y stack (justificación de los ADR)	54
10.4. Matriz de mercado laboral vs. estilo y stack	55
11. Referencias y anexos	58

Índice de figuras

1. Arquitectura de alto nivel de AeroReserva	25
2. Diagrama C4 de contexto de AeroReserva	26
3. Diagrama C4 de contenedores de AeroReserva	27
4. Diagrama C4 de componentes de Booking Service	27
5. Diagrama C4 de componentes de Flight Service	28
6. Diagrama C4 de componentes de Passenger Service	28

7.	Diagrama C4 dinámico de creación de una reserva	29
8.	Diagrama C4 de despliegue local de AeroReserva	30

Índice de cuadros

1.	Comparación entre monolito tradicional, SOA tradicional, monolito distribuido y microservicios.	6
2.	Problemas y desventajas de la arquitectura de microservicios por dimensión afectada.	7
3.	Ventajas y desventajas de gRPC según categoría de impacto.	14
4.	Comparación de alternativas tecnológicas por capa arquitectónica y decisión final para AeroReserva.	21
5.	Alcance funcional de AeroReserva	23
6.	Requisitos funcionales de AeroReserva	23
7.	Atributos de calidad de AeroReserva según ISO/IEC 25010:2023	23
8.	Componentes principales de la arquitectura de AeroReserva	25
9.	Mensaje Booking (el dato central de este servicio)	31
10.	Códigos de error comunes a los tres microservicios	32
11.	Endpoints expuestos por el BFF	33
12.	Espacios de datos (<i>keyspaces</i>) de AeroReserva	35
13.	Matriz de ejecución y verificación de los contenedores de AeroReserva	39
14.	Riesgos operativos y de despliegue de AeroReserva	39
15.	Atributos de calidad evaluados en AeroReserva	41
16.	Patrones de diseño GoF implementados en AeroReserva	44
17.	Matriz de atributos de calidad (ISO/IEC 25010) frente al estilo de microservicios	50
18.	Matriz de principios de diseño frente al estilo de microservicios	52
19.	Tácticas arquitectónicas aplicadas en cada ADR de AeroReserva	54
20.	Mercado laboral (Estados Unidos) para el estilo de microservicios y cada tecnología del stack de AeroReserva	55

Resumen ejecutivo

AeroReserva es una aplicación web de demostración para consultar vuelos, identificar pasajeros y crear, consultar y cancelar reservas. Su arquitectura divide estas funciones entre tres microservicios: Vuelos administra el catálogo y los cupos; Pasajeros gestiona los datos de identificación; y Reservas coordina el proceso de reserva. El frontend, desarrollado con Astro, actúa también como *Backend For Frontend* (BFF) y se comunica con los servicios mediante gRPC. Cada servicio utiliza su propio *keyspace* en una instancia de Cassandra, mientras Docker Compose permite iniciar el sistema completo en un entorno local. El proyecto demuestra cómo colaboran servicios con responsabilidades separadas y cómo se recupera un cupo si falla la creación de una reserva. La identificación de pasajeros se implementó únicamente para la demostración y no equivale a un sistema de autenticación de producción.

1. Investigación

1.1. Microservicios

La arquitectura de microservicios es un estilo de diseño de sistemas distribuidos que estructura una aplicación como un conjunto de servicios autónomos, de grano fino, independientemente desplegables y acoplados de forma débil. Cada microservicio encapsula una capacidad de negocio específica y delimita sus datos y su lógica mediante dominios conceptuales claramente definidos. La comunicación entre los servicios se efectúa a través de interfaces de programación de aplicaciones (API) agnósticas y protocolos livianos.

Para comprender el alcance de los microservicios resulta indispensable analizar sus delimitaciones frente a otros paradigmas arquitectónicos con los que con frecuencia se confunden.

Aunque la arquitectura de microservicios deriva conceptualmente de la Arquitectura Orientada a Servicios (SOA), existen diferencias fundamentales en su implementación y filosofía. La SOA tradicional empresarial de la década de 2000 se articuló en torno a un Bus de Servicios Empresariales (*Enterprise Service Bus*, ESB) altamente complejo y centralizado, que concentraba la lógica de enrutamiento, transformación de protocolos y orquestación de negocio.

Por otro lado, es fácil caer en el antipatrón del **monolito distribuido**. Este representa un antipatrón de diseño crítico en el que un sistema se divide físicamente en múltiples ejecutables distribuidos en red, pero mantiene un alto acoplamiento temporal, funcional y de datos. Si el servicio A no puede funcionar sin la disponibilidad síncrona e inmediata del servicio B, o si múltiples servicios comparten directamente las tablas de una misma base de datos, el sistema hereda los peores atributos de ambos mundos: la complejidad operativa y la latencia

de un sistema distribuido, combinadas con el acoplamiento de un monolito.

Cuadro 1: Comparación entre monolito tradicional, SOA tradicional, monolito distribuido y microservicios.

Dimensión arquitectónica	Monolito tradicional	SOA tradicional	Monolito distribuido	Microservicios
Unidad de despliegue	Artefacto único.	Múltiples servicios de grano grueso.	Múltiples artefactos acoplados en red.	Múltiples artefactos de grano fino e independientes.
Integración de datos	Base de datos centralizada y compartida.	Tendencia a base de datos compartidas o integradas vía ESB.	Base de datos compartida.	Base de datos dedicada por servicio.
Capa de comunicación	Llamadas a métodos en memoria.	Bus de servicios empresariales con protocolos pesados (SOAP).	RPC síncrono acoplado o llamadas de red frágiles.	Protocolos livianos (REST, gRPC, mensajería asíncrona).
Lógica de integración	Interna en el código de dominio.	Concentrada centralmente en el ESB.	Dispersa pero dependiente del estado síncrono de otros nodos.	<i>Endpoint</i> inteligente; la red es un canal de transporte.
Gobernanza tecnológica	Monolítica y homogénea.	Estandarizada a nivel corporativo.	Heterogénea en apariencia, acoplada en la práctica.	Descentralizada y polígota.
Autonomía de despliegue	Ninguna (requiere despliegue total).	Media (sujeta a pruebas de regresión corporativas).	Nula (despliegues coordinados en cascada).	Total (despliegue independiente por servicio).

1.1.1. Historia

La historia del estilo arquitectónico de microservicios surge como una evolución pragmática de la industria para superar los límites de escalabilidad impuestos por las arquitecturas monolíticas en empresas de hipercrecimiento. Sus raíces teóricas se remontan al año 2003, con el trabajo de Eric Evans sobre el Diseño Guiado por el Dominio (DDD), el cual proporcionó la base conceptual para la descomposición de servicios a través de contextos delimitados. Posteriormente, entre 2006 y 2009, la comunidad web migró hacia arquitecturas orientadas a recursos (REST sobre HTTP) y sistemas dirigidos por eventos, debido al fracaso operativo y la sobrecomplejidad de la arquitectura SOA empresarial tradicional. El verdadero punto de inflexión práctico ocurrió en 2008, cuando Netflix sufrió un fallo catastrófico en su base de datos, lo que impulsó su migración masiva a la nube de AWS y la desintegración pionera de su monolito en cientos de servicios distribuidos. Tras la liberación de las herramientas de código abierto de Netflix entre 2011 y 2012, el término *microservice* se discutió en 2012 y fue formalizado teóricamente en 2014 por Martin Fowler y James Lewis. Finalmente, la consolidación

de Docker y Kubernetes entre 2015 y 2017 eliminó las barreras operativas para empaquetar contenedores a escala, mientras que, a partir de 2020, empresas como Uber comenzaron a evolucionar el paradigma hacia enfoques jerárquicos como DOMA (*Domain-Oriented Microservice Architecture*) para gestionar la proliferación descontrolada de servicios.

1.1.2. Características

- **Alta cohesión y bajo acoplamiento:** cada microservicio debe modelarse alrededor de un contexto delimitado.
- **Descentralización de datos y gobernanza políglota:** cada microservicio selecciona la tecnología de almacenamiento optimizada para sus patrones de acceso, como PostgreSQL para transacciones relacionales, Cassandra para escrituras masivas de series temporales o Neo4j para relaciones de grafos.
- **Autonomía de despliegue y ejecución:** un microservicio debe poder compilarse, probarse, desplegarse y escalarse de forma independiente, sin requerir la coordinación ni la pausa de otros servicios del ecosistema.
- **Diseño para el fallo:** la arquitectura asume que los componentes fallarán de manera recurrente. Por ello integra mecanismos de tolerancia a fallos a nivel de infraestructura y aplicación para prevenir el colapso en cascada del sistema total.
- **Automatización extrema de la infraestructura:** la contenedorización, la orquestación automatizada y la observabilidad distribuida son prerequisites indispensables para la sostenibilidad del estilo.

1.1.3. Problemas y desventajas

Cuadro 2: Problemas y desventajas de la arquitectura de microservicios por dimensión afectada.

Dimensión afectada	Problema	Desventaja
Rendimiento y comunicación	Sustitución de las llamadas a métodos en memoria por llamadas a través de la red.	Introduce latencia adicional y genera costos computacionales asociados a la serialización y deserialización de datos.
Consistencia de datos	Eliminación de las transacciones ACID distribuidas y falta de transacciones transversales entre servicios.	Exige adoptar el modelo BASE (consistencia eventual), lo cual requiere implementar patrones complejos como Saga y gestionar explícitamente los estados intermedios.

Dimensión afectada	Problema	Desventaja
Operatividad e infraestructura	La necesidad de gestionar decenas o cientos de servicios de ejecución independientes en un entorno distribuido.	Requiere una madurez organizacional elevada para implementar y mantener herramientas de monitoreo distribuido, registro centralizado, orquestación de red y gestión de secretos.
Organización (Ley de Conway)	La arquitectura del sistema debe reflejar las líneas de comunicación de la empresa.	Si no se reestructura a los equipos de ingeniería en unidades autónomas y multifuncionales, se generan fricciones de coordinación masivas.
Pruebas (testabilidad)	Las dependencias de red distribuida dificultan el entorno de pruebas a escala global.	Hace que las pruebas de integración sean altamente complejas, a pesar de que las pruebas de unidad o de API individuales sean simples.
Resiliencia e integración	Las solicitudes fragmentadas de los clientes y la dependencia en servicios lentos o degradados.	Riesgo de latencia excesiva y colapsos en cascada, lo que obliga a implementar patrones adicionales (como API Gateway, Circuit Breaker o Bulkhead) para aislar fallos y enrutar peticiones.
Escalado organizacional (<i>sprawl</i>)	La proliferación descontrolada de servicios (<i>microservice sprawl</i>) que crea redes de dependencias desestructuradas (grafos $N \times M$).	Incrementa drásticamente la latencia por llamadas encadenadas, dificulta la trazabilidad y provoca fallos impredecibles (como evidenció el caso de Uber antes de migrar a DOMA).

1.1.4. Patrones y cuándo usarlos

Integración y comunicación.

- **API Gateway:** se debe aplicar en casi todos los ecosistemas de microservicios que cuenten con clientes externos. Su uso centraliza la gestión de políticas transversales y la agregación de respuestas ante peticiones fragmentadas.
- **Backend-For-Frontend (BFF):** se utiliza para evitar que ciertos clientes (como las aplicaciones móviles) procesen datos innecesarios, aplicándose cuando se requiere crear pasarelas optimizadas de forma específica para cada tipo de cliente (móvil, web, dispositivos IoT).

Gestión de datos y consistencia.

- **Database per Service:** su aplicación es obligatoria para garantizar la autonomía de los componentes y aislar la persistencia de los datos.
- **Saga:** se debe aplicar cuando existen procesos de negocio de múltiples pasos que abarcan varios servicios, permitiendo coordinar la secuencia de transacciones sin recurrir a

bloqueos de dos fases.

- **CQRS**: se aplica específicamente en dominios que presentan una alta disparidad entre sus volúmenes de lectura y escritura, permitiendo que cada modelo opere y se optimice de manera independiente.
- **Event Sourcing**: es ideal utilizarlo en sistemas financieros, sistemas de auditoría crítica o dominios basados en eventos, donde es necesario mantener la trazabilidad y un registro inmutable como fuente primaria de verdad.

Resiliencia.

- **Circuit Breaker**: se debe implementar de manera obligatoria en todas las interacciones interservicio que sean síncronas a través de la red, para interrumpir llamadas y evitar que la latencia excesiva genere un colapso en cascada.
- **Bulkhead**: se utiliza para aislar recursos específicos del sistema (como conexiones a bases de datos o hilos de ejecución), garantizando que el fallo de una dependencia no termine agotando los recursos del servicio que la invocó.

Migración y modernización de sistemas.

- **Strangler Fig**: se aplica en escenarios de modernización de sistemas monolíticos heredados que están en producción, utilizándose para reemplazar funcionalidades de manera gradual y así evitar el riesgo inaceptable de realizar migraciones masivas de tipo *Big Bang*.

Consolidación y jerarquización.

- **DOMA (*Domain-Oriented Microservice Architecture*)**: se debe aplicar de forma estratégica cuando la proliferación descontrolada de servicios (*microservice sprawl*) aumenta la latencia, degrada la observabilidad y genera interacciones desestructuradas. Se usa agrupando los servicios en dominios de negocio, implementando capas jerárquicas estrictas (donde las llamadas solo pueden fluir hacia abajo) y forzando a que las llamadas externas interactúen únicamente a través de un *Gateway* de dominio.

1.1.5. Ejemplos de la industria

Un ejemplo pionero en la industria es el de Netflix, que en 2008 operaba un servicio de alquiler y transmisión de vídeo soportado por un monolito relacional en sus propios centros

de datos. Tras un fallo catastrófico en su base de datos que paralizó las operaciones por tres días, la empresa inició una migración hacia la nube de AWS, desintegrando el monolito en cientos de microservicios para poder escalar componentes de forma independiente y aislar fallos a nivel de región. Otro caso destacable es el de Uber, cuya plataforma evolucionó hasta alcanzar más de 4.000 microservicios, lo cual generó una red de dependencias desestructurada que incrementó drásticamente la latencia y provocó fallos en cascada. Para solventar esta proliferación descontrolada, Uber implementó una arquitectura denominada DOMA (*Domain-Oriented Microservice Architecture*), reestructurando su sistema mediante dominios lógicos, jerarquías de capas estrictas y *gateways* de dominio, lo que redujo la complejidad y los costos de soporte operativo.

1.2. Astro

1.2.1. Definición: qué es y qué no es

Astro es un *framework* para construir sitios y aplicaciones web. Su propuesta consiste en generar HTML y enviar JavaScript al navegador únicamente cuando una parte de la página necesita interacción. No es una base de datos, un protocolo de comunicación ni una arquitectura de microservicios. Tampoco es exclusivamente un generador de sitios estáticos: puede producir páginas dinámicas y ejecutar rutas de API en un servidor (Astro, s.f.-g).

1.2.2. Características principales

Astro organiza las páginas mediante archivos, permite crear componentes `.astro` y utiliza la arquitectura de islas: la mayor parte de la página puede entregarse como HTML, mientras los componentes que requieren interacción cargan JavaScript de forma independiente. También admite componentes de React, Vue y otros *frameworks*, además de rutas de API y renderizado bajo demanda mediante un adaptador (Astro, s.f.-b, s.f.-c, s.f.-g).

1.2.3. Historia y evolución

Astro se presentó en 2021 como una herramienta para construir sitios rápidos enviando menos JavaScript al navegador. La versión 1.0 llegó en 2022 con soporte estable para renderizado en servidor. En 2023, Astro 2.0 incorporó colecciones de contenido y la posibilidad de combinar páginas estáticas y dinámicas. Astro 5.0 amplió esa combinación mediante funciones como las *islas de servidor*, que permiten mostrar contenido dinámico dentro de páginas mayormente estáticas (Astro, s.f.-b, s.f.-g).

1.2.4. Ventajas y desventajas

Su principal ventaja es el control sobre el JavaScript que recibe el navegador. Además, permite desarrollar páginas, componentes y rutas de servidor dentro de un mismo proyecto. Como desventaja, su enfoque orientado al contenido aporta menos beneficios si casi toda la interfaz es altamente interactiva y comparte estado continuamente. Asimismo, las páginas dinámicas requieren configurar y mantener un entorno de ejecución en el servidor. Esta valoración sobre las interfaces muy interactivas es una inferencia basada en el enfoque de Astro; no significa que Astro sea incapaz de construirlas (Astro, s.f.-b, s.f.-c, s.f.-d).

1.2.5. Casos de uso: cuándo usarlo y cuándo no

Astro resulta adecuado para sitios de documentación, blogs, catálogos y aplicaciones cuyas páginas combinan información visible con interacciones puntuales. En cambio, si la mayor parte de la pantalla funciona como una aplicación interactiva continua, conviene compararlo con *frameworks* centrados en ese modelo antes de seleccionarlo. En AeroReserva se utiliza para mostrar vuelos, formularios y reservas, y para recibir las solicitudes del navegador mediante rutas de servidor (Astro, s.f.-a, s.f.-c, s.f.-d).

1.2.6. Casos de aplicación en la industria

Microsoft empleó Astro en el sitio de su sistema de diseño Fluent 2, donde buscaba presentar contenido y ejemplos mediante una solución ligera compatible con sus componentes existentes (Astro, s.f.-f). Firebase lo utilizó para reconstruir su blog; el caso publicado por Astro informa mejoras de rendimiento y una reducción del tiempo necesario para publicar contenido (Astro, s.f.-e). Estos casos demuestran el uso empresarial de **Astro**, pero no constituyen evidencia de que dichas organizaciones utilicen el *stack* completo de AeroReserva.

1.2.7. Relación con el estilo de microservicios y las tecnologías seleccionadas

Astro ocupa la capa que interactúa con el viajero. En AeroReserva, sus rutas de servidor reciben las solicitudes web y las adaptan para comunicarse con los servicios de Vuelos, Pasajeros y Reservas. Los servicios implementan las reglas de negocio y conservan sus datos por separado. Por tanto, Astro facilita la interfaz y la entrada al sistema, pero **no convierte por sí solo una aplicación en microservicios**: esa separación depende de los límites y responsabilidades definidos para cada servicio. El uso de rutas de API y renderizado en servidor está respaldado por las capacidades de Astro; su conexión concreta con Ruby y gRPC es una decisión del proyecto (Astro, s.f.-c, s.f.-g).

1.2.8. Frecuencia de uso del *stack* combinado

Astro aparece en estudios del ecosistema JavaScript y cuenta con casos de uso documentados. Sin embargo, esas fuentes miden o describen **Astro por separado**: no permiten determinar cuántos proyectos combinan específicamente Astro, Ruby, gRPC, Cassandra y Docker Compose. En consecuencia, no sería riguroso afirmar que la combinación completa es común ni asignarle un porcentaje. Su elección en AeroReserva se explica por la función de cada pieza dentro del sistema, no por una estadística de adopción conjunta (State of JS, 2025).

1.3. gRPC

gRPC (*Google Remote Procedure Call*) es un marco de trabajo de llamadas a procedimientos remotos, de código abierto y de alto rendimiento, diseñado por Google y custodiado por la *Cloud Native Computing Foundation* (CNCF). El paradigma fundamental de gRPC se basa en una arquitectura orientada a contratos (*contract-first API design*) sobre el protocolo de transporte HTTP/2, utilizando *Protocol Buffers* (Protobuf) como lenguaje de definición de interfaz (IDL) y mecanismo de serialización binaria. En este modelo, un cliente invoca directamente métodos en un servidor remoto como si se tratara de una ejecución funcional local, abstrayendo la complejidad de la red mediante *stubs* fuertemente tipados generados automáticamente.

Para delimitar el alcance de gRPC en la arquitectura de software, es necesario analizar sus diferencias conceptuales frente a otras alternativas de comunicación:

- **Diferencia con REST sobre HTTP/1.1:** en comparación con REST sobre HTTP/1.1, REST es un estilo arquitectónico orientado a recursos que utiliza verbos HTTP estándar (GET, POST, PUT, DELETE) sobre cargas de pago en texto plano o semiestructurado, como JSON o XML, y carece de un esquema estricto por defecto, mientras transmite errores mediante los códigos de estado HTTP nativos. Por el contrario, gRPC se orienta a acciones o procedimientos, transporta cargas binarias no autodescriptivas y exige un contrato estricto precompilado mediante archivos `.proto`. Además, gRPC encapsula el estado de la ejecución en encabezados de cierre (*trailers*) de HTTP/2 (`grpc-status`), devolviendo casi siempre un código de transporte HTTP 200 OK a nivel de red.
- **Diferencia con WebSockets:** frente a WebSockets, esta última tecnología establece un canal de comunicación dúplex sobre una única conexión TCP en la capa de aplicación, pero no define una semántica de mensajería, esquemas de datos ni enrutamiento,

enviando marcos de texto o binarios crudos sin tipado de datos. En cambio, gRPC proporciona un marco de trabajo completo que integra tipado estricto, serialización, multiplexación de solicitudes, propagación de contexto, plazos de tiempo (*deadlines*) y cancelación remota mediante flujos sobre HTTP/2.

- **Diferencia con GraphQL:** en relación con GraphQL, este se diseña como un lenguaje de consultas para API del lado del cliente, enfocado en evitar el sobreabastecimiento o subabastecimiento de datos mediante la agregación flexible en un único punto de enlace (*endpoint*) HTTP. Por su parte, gRPC está pensado para la comunicación directa entre servicios *backend* de baja latencia, donde se prioriza la velocidad de serialización y el consumo eficiente de memoria y CPU por encima de la flexibilidad de consulta del cliente.

1.3.1. Historia

gRPC nació a principios de la década de 2010 a partir de Stubby, un sistema propietario de llamadas a procedimientos remotos desarrollado por Google para procesar cientos de miles de millones de RPC internas por segundo. Debido a que Stubby estaba fuertemente acoplado a la infraestructura interna de Google, la compañía rediseñó su arquitectura en 2015 para aprovechar el estándar abierto HTTP/2 y *Protocol Buffers*, convirtiéndolo en la herramienta agnóstica de código abierto conocida como gRPC. Tras ser donado a la *Cloud Native Computing Foundation* (CNCF), el marco de trabajo evolucionó incorporando tecnologías clave como gRPC-Web para la integración con navegadores, integración nativa con mallas de servicios (*Service Meshes*) mediante el protocolo xDS, soporte para HTTP/3 (QUIC), y la transición de esquemas de sintaxis heredados (proto2/proto3) hacia el modelo unificado de *Protocol Buffers Editions*.

1.3.2. Características

- **Arquitectura orientada a contratos (*contract-first*):** se basa en la definición estricta de esquemas y servicios en archivos `.proto` antes de la implementación, actuando como la única fuente de verdad funcional.
- ***Protocol Buffers* (Protobuf) como IDL y serialización:** utiliza un formato binario denso que codifica mediante etiquetas numéricas (*field tags*) y esquemas *Tag-Length-Value* (TLV) con *Varint*. Esto elimina la sobrecarga de metadatos repetitivos (como nombres de claves en JSON) y optimiza el uso de CPU y memoria.

- **Multiplexación:** envío simultáneo de múltiples solicitudes y respuestas intercaladas sobre una sola conexión TCP, evitando el bloqueo de cabeza de línea.
- ***Bi-directional streaming:*** flujos de mensajes independientes de lectura y escritura simultánea entre ambos extremos.
- ***Client Streaming RPC:*** una secuencia de mensajes enviados por el cliente y una sola respuesta final del servidor.
- ***Server Streaming RPC:*** una solicitud del cliente y una secuencia (*stream*) continua de respuestas del servidor.
- ***Unary RPC:*** solicitud y respuesta únicas.

1.3.3. Ventajas y desventajas

Cuadro 3: Ventajas y desventajas de gRPC según categoría de impacto.

Categoría de desventaja	Descripción técnica / punto de dolor	Impacto arquitectónico y operativo
Complejidad de depuración	Las tramas de datos transmitidas por la red están serializadas en formato binario denso (Protobuf), lo que las hace ilegibles para el ojo humano.	Requiere herramientas especializadas (como Wireshark o <code>grpcurl</code>), interceptores y la posesión previa de los archivos <code>.proto</code> compilados para poder inspeccionar o depurar el tráfico.
Soporte limitado en navegadores	Las API nativas de los navegadores web modernos no ofrecen control granular sobre las tramas HTTP/2 ni acceso directo a los encabezados de cierre (<i>trailers</i>) de red.	Obliga a implementar una capa de abstracción (gRPC-Web) y desplegar proxies de traducción (como Envoy), añadiendo complejidad operacional a la infraestructura <i>frontend</i> .
Curva de aprendizaje elevada	Requiere un conocimiento profundo del funcionamiento interno de HTTP/2, gestión de canales multiplexados y estrategias de balanceo de carga en capa 7.	Aumenta la complejidad del ciclo de desarrollo al exigir la configuración y mantenimiento continuo de cadenas de compilación (<code>protoc</code> / <code>Buf</code>).
Gobernanza de esquemas estricta	El mantenimiento de compatibilidad hacia atrás exige seguir reglas estrictas (no reasignar <i>field tags</i> , declarar campos <i>reserved</i> , etc.).	Cualquier modificación incorrecta en los contratos <code>.proto</code> puede provocar fallos de compatibilidad a nivel de red (<i>wire</i>), de paquetes (<i>package</i>) o de código compilado (<i>file</i>).
Sobrecarga en escenarios simples	Para sistemas CRUD sencillos de baja frecuencia o API públicas para terceros, gRPC resulta excesivo.	Introduce una sobrecarga de configuración que no compensa los beneficios de rendimiento en dominios donde el procesamiento de texto JSON/REST no es un cuello de botella.

1.3.4. Casos de uso

- **Malla de microservicios *backend* (ideal)**: para la comunicación interna *backend-to-backend* de alta densidad, donde minimiza la latencia, reduce el consumo de CPU mediante serialización binaria y garantiza contratos tipados.
- ***Streaming* e ingesta de datos en tiempo real (ideal)**: para transferencias continuas de datos, ya que el soporte nativo de *streaming* bidireccional sobre conexiones multiplexadas elimina la sobrecarga de conexiones repetitivas.
- **Entornos IoT y redes restringidas (ideal)**: para dispositivos con recursos limitados o enlaces inestables, debido a que el formato binario denso de Protobuf maximiza la transferencia útil de datos.
- **Plano de control e infraestructura distribuida**: para la replicación de datos, algoritmos de consenso e interfaces internas en plataformas como Kubernetes, CockroachDB o mallas de servicios.

1.3.5. Casos en la industria

En la industria, grandes empresas de tecnología utilizan gRPC para optimizar la comunicación de alta velocidad entre sus sistemas. **Google** impulsó su creación a partir de su infraestructura interna Stubby para gestionar cientos de miles de millones de llamadas por segundo entre sus centros de datos, mientras que **Netflix** migró sus microservicios de REST/JSON a gRPC para reducir sustancialmente el uso global de CPU y acelerar la respuesta en su plataforma de *streaming*. Asimismo, **Square** lo implementó en sus sistemas de procesamiento de pagos para garantizar un tipado estricto de datos en tiempo de compilación y evitar fallos en transacciones financieras críticas.

1.3.6. Relación entre el estilo y gRPC

La relación entre la arquitectura de microservicios y gRPC es fundamentalmente una alianza orientada al alto rendimiento y la eficiencia en la red. En un sistema de microservicios, una sola petición de usuario suele desencadenar decenas de llamadas interservicio (*backend-to-backend*). gRPC resuelve los cuellos de botella de esta comunicación distribuida gracias al uso de transporte sobre HTTP/2 y la serialización binaria con *Protocol Buffers*. Esto reduce drásticamente el consumo de CPU y ancho de banda en comparación con el formato JSON, mientras que la multiplexación permite enviar múltiples solicitudes simultáneas sobre una única conexión TCP, minimizando la latencia global del sistema.

1.3.7. Qué tan común es el *stack* designado

La combinación de Astro, Ruby on Rails y gRPC es inusual y sobreingeniada, porque gRPC no es apto para el navegador desde donde opera Astro —al requerir *proxies* como gRPC-Web para superar las limitaciones de las API de red del cliente— ni tampoco es la opción natural para un *framework* monolítico en texto plano como Rails; por ello, en una arquitectura realista, Astro se comunicaría con Rails mediante REST/JSON o GraphQL, dejando a gRPC relegado únicamente como cliente en Rails para realizar llamadas internas de alta velocidad hacia microservicios *backend* desarrollados en lenguajes como Go, C++ o Rust.

1.4. Ruby on Rails

1.4.1. Definición: qué son y qué no son

Ruby es un lenguaje de programación orientado a objetos con una sintaxis diseñada para favorecer la lectura del código. Ruby on Rails, o Rails, es un *framework* construido con Ruby que proporciona una estructura y convenciones para desarrollar aplicaciones web. No son equivalentes: un programa puede estar escrito en Ruby sin utilizar Rails. Ninguno de los dos es, por sí mismo, un protocolo de comunicación o una arquitectura de microservicios (Ruby, s.f.; Ruby on Rails, s.f.-b).

1.4.2. Características principales

Ruby permite organizar la lógica mediante clases y módulos, y ampliar sus capacidades con bibliotecas llamadas *gems*. Rails reúne herramientas y convenciones para tareas habituales del desarrollo web, con el principio de «convención sobre configuración»: establece valores predeterminados para reducir decisiones repetitivas. Ruby también puede ejecutar procesos independientes de una aplicación web; por ejemplo, un servidor gRPC cuya interfaz se genera a partir de un archivo `.proto` (Ruby, s.f.; Ruby on Rails, s.f.-b; gRPC, s.f.-c).

1.4.3. Historia y evolución

Yukihiro Matsumoto comenzó a desarrollar Ruby en 1993 y lo publicó en 1995. Con el tiempo se formó un ecosistema de bibliotecas y herramientas alrededor del lenguaje. Rails apareció en 2004 y difundió un modelo de desarrollo web basado en convenciones. Ambos proyectos han seguido evolucionando, pero conservan funciones distintas: Ruby proporciona el lenguaje de programación y Rails, una forma de estructurar aplicaciones web (Ruby, s.f.; Ruby on Rails, s.f.-a).

1.4.4. Ventajas y desventajas

Ruby favorece la legibilidad y dispone de bibliotecas para integrar pruebas, bases de datos y comunicación entre servicios. Rails puede acelerar la organización de una aplicación web porque ofrece una estructura y decisiones predeterminadas. Como contrapartida, incorporar un *framework* amplio a un servicio pequeño puede añadir componentes y dependencias que ese servicio no necesita. Además, el código Ruby exige atención a su rendimiento cuando la carga y los tiempos de respuesta son requisitos exigentes; esto debe evaluarse con mediciones del caso concreto, no suponerse a partir del lenguaje (Ruby, s.f.; Ruby on Rails, s.f.-b). La valoración sobre el tamaño del *framework* es una inferencia de diseño, no una limitación que impida usar Rails en servicios pequeños.

1.4.5. Casos de uso: cuándo usarlos y cuándo no

Ruby es adecuado cuando se busca desarrollar lógica de negocio legible y se dispone de *gems* apropiadas para las integraciones necesarias. Rails resulta especialmente útil cuando una aplicación aprovecha sus convenciones y herramientas web. Si un proceso tiene una función muy limitada y no utiliza esas herramientas, conviene comparar Rails con una implementación más pequeña en Ruby. En AeroReserva, los servicios de Vuelos, Pasajeros y Reservas ejecutan lógica en Ruby y atienden solicitudes gRPC; por ello, al describirlos es necesario distinguir la estructura del proyecto de las funciones de Rails que realmente utilizan (Ruby on Rails, s.f.-b; gRPC, s.f.-a).

1.4.6. Casos de aplicación en la industria

Shopify ha documentado el uso de Ruby y Rails en partes importantes de su plataforma, así como en el *backend* de su aplicación Shop. Estos ejemplos muestran que las tecnologías pueden utilizarse en sistemas de gran escala, junto con decisiones adicionales de arquitectura y operación. No indican que Shopify utilice la combinación específica de Astro, gRPC, Cassandra y Docker Compose elegida para AeroReserva (Shopify Engineering, s.f.).

1.4.7. Relación con el estilo de microservicios y las tecnologías seleccionadas

En AeroReserva, Ruby implementa las reglas de negocio de tres servicios separados: Vuelos, Pasajeros y Reservas. *Protocol Buffers* definen sus mensajes y operaciones, y la biblioteca gRPC para Ruby permite generar las clases cliente y servidor a partir del contrato `.proto`. Cada servicio accede a los datos que le corresponden en Cassandra, mientras Astro recibe las solicitudes del navegador. Ruby y Rails no producen automáticamente una arquitectura de

microservicios: esta depende de la separación de responsabilidades, los contratos y el despliegue de los servicios. Las operaciones entre ellos se exponen mediante **gRPC**, no mediante controladores HTTP de Rails.

1.4.8. Frecuencia de uso del *stack* combinado

Los casos publicados permiten afirmar que Ruby y Rails tienen uso en la industria. Sin embargo, documentar la adopción de Ruby o Rails por separado no permite calcular qué tan frecuente es la combinación exacta **Astro + Ruby/Rails + gRPC + Cassandra + Docker Compose**. Por ello, AeroReserva justifica este *stack* según la función de cada tecnología dentro del sistema, sin atribuirle una cifra de adopción conjunta que las fuentes consultadas no proporcionan.

1.5. Cassandra

1.5.1. Definición clara: qué es y qué no es

Apache Cassandra es un sistema de gestión de bases de datos NoSQL, distribuido, de código abierto y de arquitectura *scale-out* (masivamente paralela), diseñado para manejar grandes volúmenes de datos a través de muchos nodos sin un punto único de fallo. Combina el modelo de almacenamiento en columnas de Google Bigtable con el modelo de distribución descentralizado tipo anillo de Amazon Dynamo. Su arquitectura es *masterless* (sin nodo maestro): cualquier nodo puede recibir una lectura o escritura y actuar como coordinador de esa operación.

No es un reemplazo de una base de datos relacional: Cassandra no soporta JOINS ni integridad referencial nativa; el modelo se diseña alrededor de las consultas de la aplicación (*query-driven design*), no de la normalización de entidades. Tampoco es un almacén clave-valor simple en memoria como Redis: Cassandra es un motor persistente orientado a disco, con esquemas tabulares y ordenamiento interno mediante *clustering keys*.

1.5.2. Características principales

- **Hashing consistente y topología en anillo:** cada nodo es responsable de un rango de claves del anillo, mediante un particionador (por defecto, Murmur3).
- **Protocolo Gossip:** los nodos se descubren y detectan caídas entre sí sin depender de un coordinador central.
- **Factor de replicación configurable por *datacenter*** (`NetworkTopologyStrategy`), permitiendo replicación multirregión nativa.

- **Escritura optimizada tipo *LSM-tree***: cada escritura va a un *CommitLog* secuencial en disco (durabilidad) y a una *Memtable* en memoria; cuando la *Memtable* se llena, se vuelca a disco como un archivo inmutable (*SSTable*).
- **Consistencia ajustable (*Tunable Consistency*)**: Cassandra es un sistema AP según el teorema CAP, pero permite elegir por operación el nivel de consistencia (ONE, QUORUM, LOCAL_QUORUM, ALL).
- ***Lightweight Transactions* (LWT)**, basadas en un protocolo tipo Paxos, para operaciones condicionales (IF NOT EXISTS, IF campo = valor) cuando se necesita atomicidad sobre una fila.

1.5.3. Historia y evolución

Cassandra fue creada originalmente en Facebook en 2007-2008 por Avinash Lakshman (coautor del *paper* de Amazon Dynamo) y Prashant Malik, para resolver el problema de escalabilidad de la búsqueda en bandeja de entrada de Facebook. Facebook la liberó como código abierto en 2008; en 2009 ingresó a la *Apache Software Foundation*, y en 2010 se convirtió en proyecto de primer nivel de Apache. Las versiones 4.x se enfocan en estabilidad de nivel empresarial; la versión 5.0 incorporó búsqueda vectorial nativa (relevante para aplicaciones de IA/RAG) e índices *Storage-Attached* (SAI), más eficientes que los índices secundarios clásicos. Como reacción a los costos operativos de una base de datos escrita en Java (pausas del *garbage collector*), surgió ScyllaDB, una reimplementación de Cassandra en C++ compatible con su protocolo.

1.5.4. Ventajas y desventajas

Ventajas: escalabilidad horizontal lineal (agregar nodos aumenta capacidad sin ventanas de mantenimiento); alta disponibilidad real, sin nodo maestro único que sea punto de fallo; rendimiento de escritura excepcional gracias al diseño de *CommitLog* + *Memtable*; y replicación multirregión pensada desde el diseño original.

Desventajas: el modelado de datos exige conocer de antemano todos los patrones de consulta de la aplicación (un requisito nuevo de negocio que necesite filtrar por un campo no indexado puede obligar a crear y duplicar datos en una tabla nueva); no soporta JOINS ni agregaciones *ad hoc* complejas; el manejo de borrados mediante *tombstones* puede degradar el rendimiento de lectura si se acumulan demasiados; y mantener un clúster sano (reparaciones periódicas, ajuste de la JVM) exige un nivel de operación considerable.

1.5.5. Casos de uso: cuándo usarla y cuándo no

Cassandra es adecuada para ingesta masiva de series temporales (IoT, telemetría), *feeds* de actividad y mensajería, y sistemas financieros distribuidos multirregión que necesitan escrituras concurrentes con `LOCAL_QUORUM`. No es una buena opción para sistemas transaccionales relacionales tradicionales que necesitan `JOINS` y consultas *ad hoc* cambiantes, para volúmenes pequeños de datos donde una instancia bien afinada de PostgreSQL resulta suficiente, ni para proyectos en etapa de prototipado donde los patrones de consulta cambian cada semana.

1.5.6. Casos de aplicación real

- **Netflix** usa Cassandra como base de datos principal para el historial de reproducción y las recomendaciones, en clústeres activo-activo multirregión sobre AWS que permiten mover tráfico de una región a otra en minutos.
- **Apple** opera una de las implementaciones de Cassandra más grandes del mundo (más de 160 000 nodos reportados), sosteniendo metadatos de iCloud, iMessage y Apple Music.
- **Uber** almacena en Cassandra el historial de viajes y la telemetría en tiempo real de vehículos, aprovechando su capacidad de absorber picos de escritura en horas pico sin degradar la latencia.
- **Spotify** usa Cassandra para listas de reproducción y metadatos de canciones, usando `LOCAL_QUORUM` para que la edición de una lista se sienta instantánea localmente mientras se replica en segundo plano entre regiones.

1.6. Comparación de alternativas y síntesis

La selección del *stack* tecnológico para AeroReserva se realizó considerando las necesidades de una arquitectura basada en microservicios: separación de responsabilidades, comunicación eficiente entre servicios, persistencia independiente y facilidad de despliegue. Aunque existen diferentes tecnologías capaces de resolver cada necesidad, se seleccionaron aquellas que permiten demostrar los principios asociados al estilo arquitectónico elegido.

Cuadro 4: Comparación de alternativas tecnológicas por capa arquitectónica y decisión final para AeroReserva.

Capa	Alternativa considerada	Ventajas	Limitaciones	Decisión para AeroReserva
Frontend / BFF	React + Node.js	Amplio uso industrial, gran ecosistema y flexibilidad para aplicaciones SPA.	Requiere definir más componentes para implementar renderizado y estructura del proyecto.	Astro permite combinar generación eficiente de interfaz con <i>endpoints</i> del lado servidor para funcionar como BFF ligero.
Frontend / BFF	Angular	<i>Framework</i> completo con estructura definida y herramientas integradas.	Mayor complejidad inicial para un proyecto académico pequeño.	Astro reduce la complejidad del <i>frontend</i> y permite concentrar la comunicación con los servicios internos.
Comunicación entre servicios	REST	Fácil de implementar, ampliamente conocido y compatible con navegadores.	Mayor consumo de mensajes por JSON y menor rigidez del contrato entre servicios.	gRPC permite contratos tipados mediante <i>Protocol Buffers</i> y comunicación eficiente entre servicios internos.
Comunicación entre servicios	GraphQL	Permite consultas flexibles desde clientes.	Incrementa la complejidad del servidor y no está orientado principalmente a comunicación servicio a servicio.	gRPC se ajusta mejor a la comunicación interna entre microservicios.
Backend	Spring Boot	Ecosistema empresarial amplio y numerosas herramientas para sistemas distribuidos.	Mayor configuración inicial y consumo de recursos.	Ruby on Rails permite desarrollar servicios funcionales rápidamente manteniendo una estructura organizada.
Backend	Node.js + Express	Ligero y con gran disponibilidad de bibliotecas.	Requiere establecer más convenciones de arquitectura manualmente.	Rails proporciona convenciones integradas para organizar lógica, validaciones y servicios.
Persistencia	PostgreSQL	Base relacional madura, transacciones fuertes y consultas complejas.	Menor flexibilidad para distribución horizontal masiva.	Cassandra permite demostrar separación por servicio mediante <i>keyspaces</i> independientes y escalabilidad distribuida.
Persistencia	MongoDB	Modelo documental flexible y fácil adaptación de estructuras.	Puede generar decisiones adicionales sobre consistencia y modelado.	Cassandra se adapta mejor a datos distribuidos y patrones de consulta definidos.
Orquestación local	Kubernetes	Estándar para despliegues distribuidos en producción.	Complejidad elevada para un entorno académico y local.	Docker Compose permite ejecutar múltiples servicios con una configuración sencilla y reproducible.

El *stack* elegido para AeroReserva responde a las necesidades principales del estilo de

microservicios. Astro funciona como una capa de interacción con el usuario y como BFF, evitando que el navegador conozca la estructura interna de los servicios. Los microservicios desarrollados con Ruby on Rails separan las responsabilidades del negocio en tres dominios independientes: vuelos, pasajeros y reservas.

La comunicación mediante gRPC y *Protocol Buffers* establece contratos claros entre servicios, reduciendo errores de integración y permitiendo una comunicación eficiente. Cassandra complementa este diseño mediante la separación lógica de datos utilizando un *keyspace* por servicio, evitando que un servicio dependa directamente de la estructura interna de otro. Finalmente, Docker Compose permite ejecutar todos los componentes como un sistema integrado y reproducible durante el desarrollo y la demostración.

En conjunto, el *stack* seleccionado no busca representar una arquitectura productiva completa de una aerolínea, sino demostrar los conceptos fundamentales de una arquitectura de microservicios: independencia de responsabilidades, comunicación entre servicios, separación de datos y despliegue mediante contenedores. Esta elección es coherente con el alcance académico del proyecto y con los requisitos definidos para implementar un estilo arquitectónico utilizando tecnologías reales.

2. Contexto y alcance

AeroReserva parte de una necesidad básica en un sistema de reservas aéreas: permitir que una persona conozca los vuelos disponibles, se identifique y reserve un cupo sin interactuar directamente con los servicios internos ni escribir identificadores técnicos. Para estudiar una arquitectura de microservicios, el proyecto separa el catálogo y la disponibilidad de vuelos, los datos de pasajeros y la gestión de reservas. La aplicación web reúne estas funciones en un flujo que el equipo puede ejecutar y demostrar desde el navegador. Su alcance es académico y se concentra en consultar vuelos, crear y revisar reservas, cancelarlas y mostrar el cambio correspondiente en los cupos; no pretende cubrir todas las operaciones de una aerolínea.

2.1. Alcance funcional

Cuadro 5: Alcance funcional de AeroReserva

Incluido	Fuera de alcance
Consultar vuelos disponibles	Pagos
Registrar o identificar pasajero	Selección de silla
Crear, consultar y cancelar reservas	Check-in y equipaje
Controlar cupos disponibles	Autenticación productiva

2.2. Requisitos

Cuadro 6: Requisitos funcionales de AeroReserva

ID	Requisito
RF-01	Consultar el catálogo de vuelos, sus rutas, horarios y cupos disponibles.
RF-02	Filtrar vuelos por origen y destino.
RF-03	Registrar e identificar a un pasajero para la demostración.
RF-04	Crear una reserva para el pasajero identificado y un vuelo con cupos.
RF-05	Consultar únicamente las reservas asociadas al pasajero identificado.
RF-06	Cancelar una reserva propia y restituir el cupo correspondiente.
RF-07	Informar al usuario si una operación falla, sin presentar identificadores técnicos como pasos obligatorios del flujo.

2.3. Atributos de calidad — ISO/IEC 25010:2023

Cuadro 7: Atributos de calidad de AeroReserva según ISO/IEC 25010:2023

Categoría	Requisito de calidad para AeroReserva	Cómo comprobarlo
Adecuación funcional	El sistema debe completar los flujos de consulta, identificación, reserva, consulta de reservas y cancelación conforme a las reglas de cupos.	Ejecutar el flujo completo y comprobar los estados y cupos resultantes.
Eficiencia de desempeño	La consulta del catálogo debe responder en un tiempo aceptable durante la demostración local; se propone menos de 2 segundos en el 95 % de 30 consultas consecutivas .	Medir las respuestas del BFF en el entorno Docker descrito en el documento.

Categoría	Requisito de calidad para AeroReserva	Cómo comprobarlo
Compatibilidad	El frontend y los tres servicios deben intercambiar datos mediante las rutas BFF y el contrato gRPC sin errores de formato o campos faltantes.	Probar consulta, creación y cancelación a través del frontend y verificar las respuestas entre servicios.
Capacidad de interacción	Una persona debe poder seleccionar un vuelo, identificarse, reservar y consultar sus reservas desde la interfaz sin escribir UUID.	Completar el recorrido en el navegador y comprobar los mensajes de error y confirmación.
Fiabilidad	Si falla la creación de una reserva después de ocupar un cupo, el sistema debe compensar la operación para evitar un cupo perdido; tampoco debe confirmar reservas sin disponibilidad.	Ejecutar pruebas de fallo y de concurrencia y comparar cupos y reservas antes y después.
Seguridad	Una sesión alterada no debe permitir consultar o cancelar reservas de otro pasajero.	Probar dos pasajeros distintos, una cookie modificada y la cancelación de una reserva ajena.
Mantenibilidad	Cada servicio debe conservar su responsabilidad y permitir comprobar sus reglas mediante pruebas sin modificar los otros servicios.	Revisar la separación Flight/Passenger/Booking y ejecutar las pruebas de cada uno.
Flexibilidad	El sistema debe poder reconstruirse y arrancar en un entorno compatible con Docker Compose usando la configuración documentada, sin modificar el código fuente.	Ejecutar <code>docker compose up -build -d</code> y comprobar el estado de los servicios y la web.
Protección	Ante errores de reserva o cancelación, el sistema debe preservar la integridad de los cupos y evitar confirmar una operación incompleta.	Injectar un fallo en el flujo y verificar que no quede una reserva confirmada sin su cupo correspondiente.

3. Arquitectura y justificación

AeroReserva utiliza microservicios para separar tres responsabilidades del negocio que tienen reglas y datos distintos. Vuelos administra el catálogo y la disponibilidad de cupos; Pasajeros registra y consulta los datos de las personas; y Reservas crea, consulta y cancela reservas, coordinándose con los otros dos servicios cuando necesita comprobar un pasajero u ocupar o liberar un cupo. El frontend desarrollado con Astro presenta el flujo al usuario y funciona como BFF: recibe las solicitudes de la web y se comunica con los servicios mediante gRPC.

La separación permite que cada servicio tenga una responsabilidad definida y que sus reglas puedan modificarse y probarse sin concentrar toda la lógica en una sola aplicación.

Cada uno posee su propio *keyspace* en Cassandra, aunque los tres comparten una misma instancia física en el despliegue local. Docker Compose inicia los contenedores y establece sus dependencias. Esta implementación permite demostrar la colaboración entre servicios y también sus dificultades: una reserva involucra llamadas de red y requiere manejar fallos para no dejar inconsistencias entre la reserva y los cupos. El proyecto usa esta arquitectura como ejercicio académico; compartir Cassandra y ejecutar todo en una sola máquina limita el aislamiento del despliegue.

3.1. Componentes principales

Cuadro 8: Componentes principales de la arquitectura de AeroReserva

Componente	Responsabilidad	Tecnología
Frontend y BFF	Interfaz web y <i>endpoints</i> HTTP	Astro / Node
Flight Service	Vuelos y cupos	Ruby on Rails / gRPC
Passenger Service	Pasajeros	Ruby on Rails / gRPC
Booking Service	Reservas y coordinación	Ruby on Rails / gRPC
Persistencia	Datos separados por servicio	Cassandra
Ejecución local	Contenedores y dependencias	Docker Compose

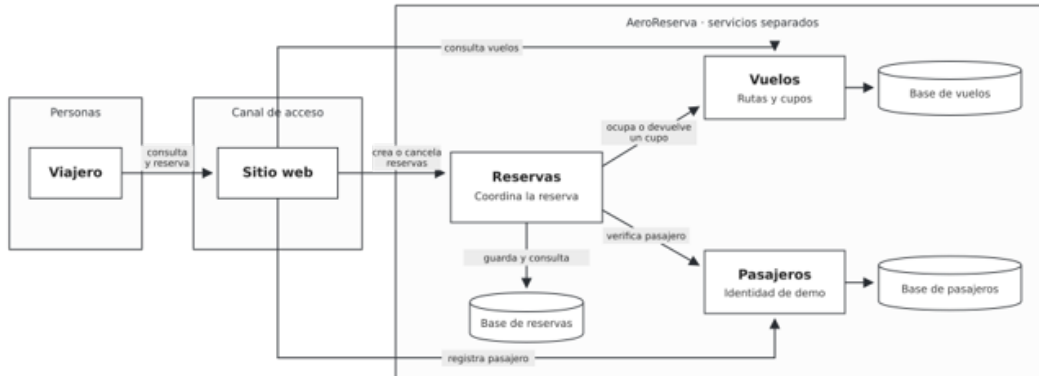


Figura 1: Arquitectura de alto nivel de AeroReserva

El diagrama de la Figura 1 representa a AeroReserva como un sistema distribuido compuesto por dos áreas principales: la interfaz que utiliza el pasajero y el conjunto de servicios que procesan las operaciones. El usuario interactúa con la aplicación web, que recibe sus solicitudes y las dirige hacia los servicios independientes de vuelos, pasajeros y reservas. Cada

servicio tiene una responsabilidad específica y administra sus propios datos, evitando concentrar toda la lógica en un único componente. Los servicios se comunican entre sí cuando una operación lo requiere, por ejemplo al crear una reserva, y finalmente consultan o actualizan sus respectivas bases de datos. Esta separación permite desarrollar, probar, mantener y escalar cada capacidad de manera independiente, que es la idea central del estilo arquitectónico de microservicios.

3.2. Diagramas C4



Figura 2: Diagrama C4 de contexto de AeroReserva

La Figura 2 muestra al pasajero como usuario del sistema: desde AeroReserva puede consultar vuelos, identificarse y crear, consultar o cancelar sus reservas. En este nivel se presenta el sistema como una unidad, sin detallar todavía sus servicios internos.

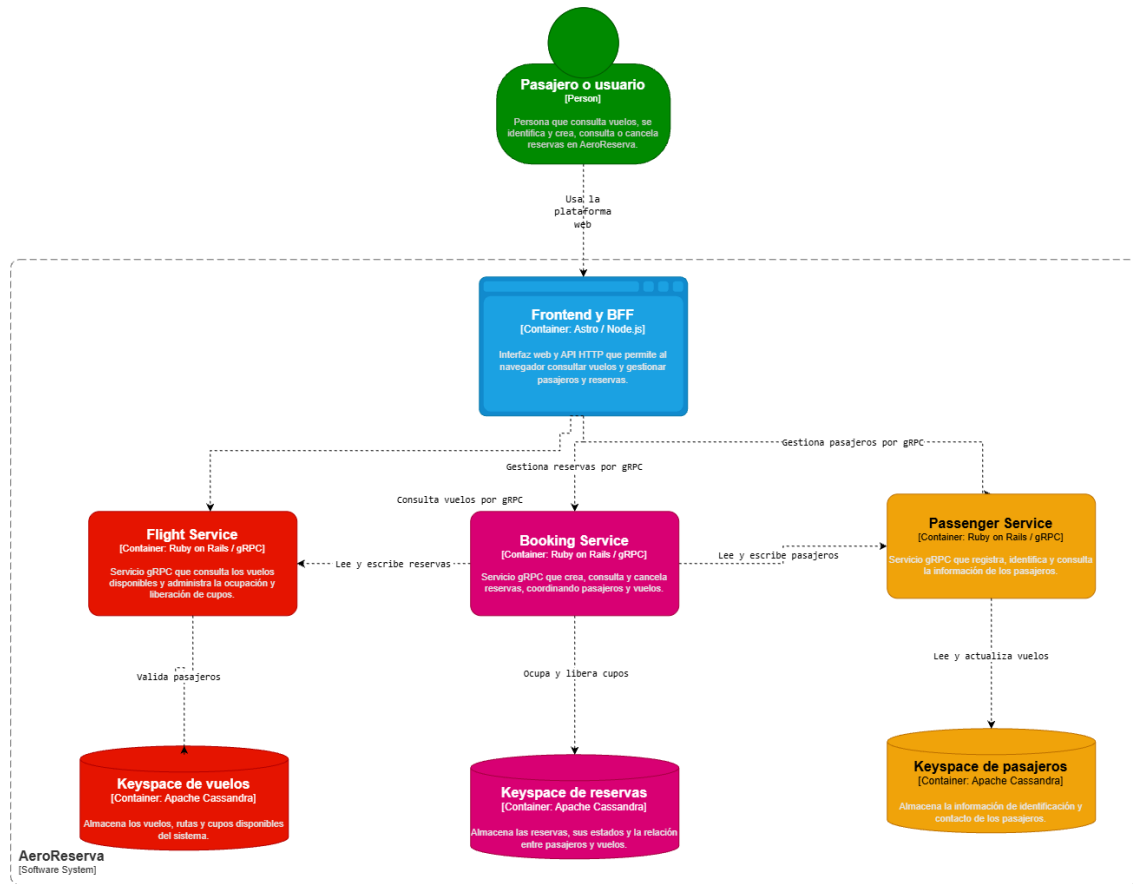


Figura 3: Diagrama C4 de contenedores de AeroReserva

La Figura 3 muestra el frontend y BFF, los tres servicios gRPC y los espacios de datos de vuelos, pasajeros y reservas. El pasajero usa la web; el BFF atiende sus solicitudes y los servicios colaboran para consultar vuelos y gestionar reservas.

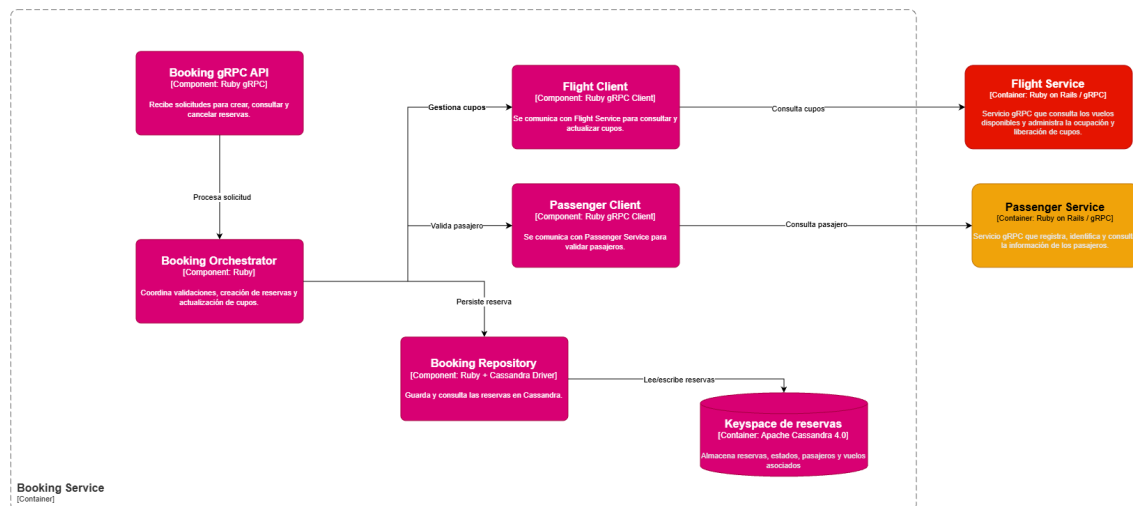


Figura 4: Diagrama C4 de componentes de Booking Service

La Figura 4 muestra cómo la API gRPC recibe las solicitudes de reserva y las entrega al orquestador. Este coordina la validación del pasajero y la gestión de cupos mediante clientes gRPC, mientras el repositorio guarda y consulta las reservas en su propio *keyspace* de Cassandra.

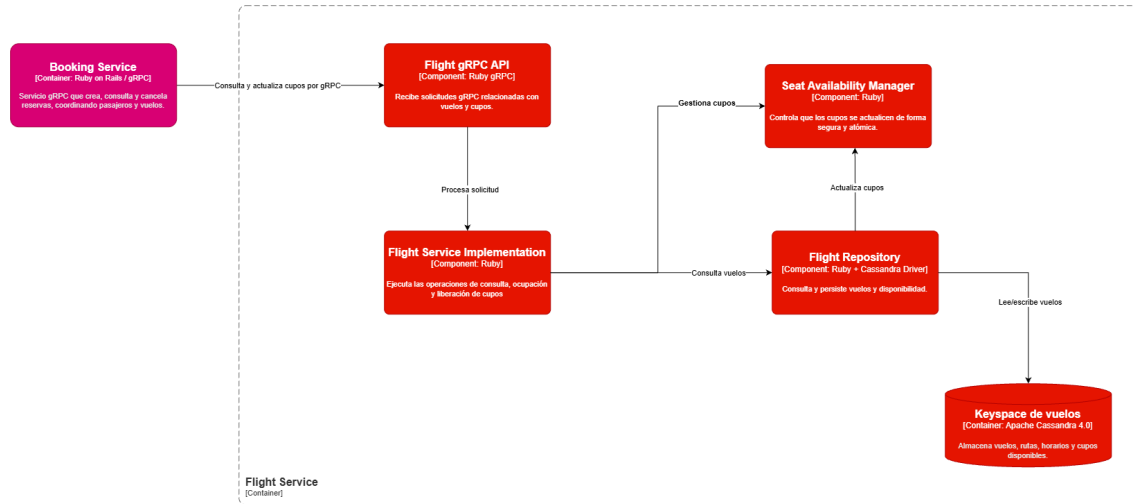


Figura 5: Diagrama C4 de componentes de Flight Service

La Figura 5 muestra cómo el servicio recibe por gRPC las consultas y solicitudes de actualización de cupos. La implementación procesa cada operación, el gestor de disponibilidad controla los cambios de cupos y el repositorio consulta o guarda la información de vuelos en el *keyspace* correspondiente de Cassandra.

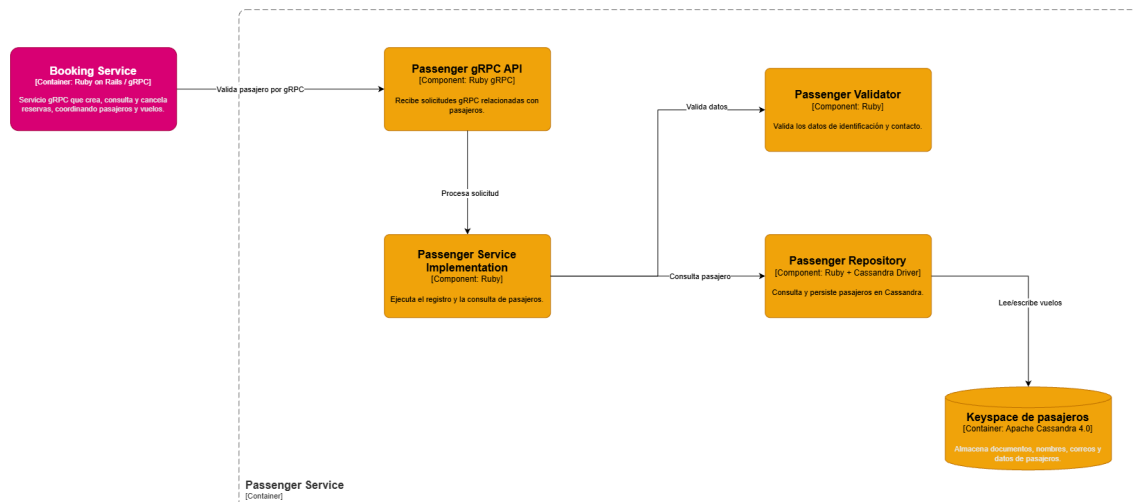


Figura 6: Diagrama C4 de componentes de Passenger Service

La Figura 6 muestra cómo la API gRPC recibe las solicitudes sobre pasajeros: la implementación ejecuta el registro o la consulta, el validador comprueba los datos y el repositorio

accede al *keyspace* de pasajeros en Cassandra.

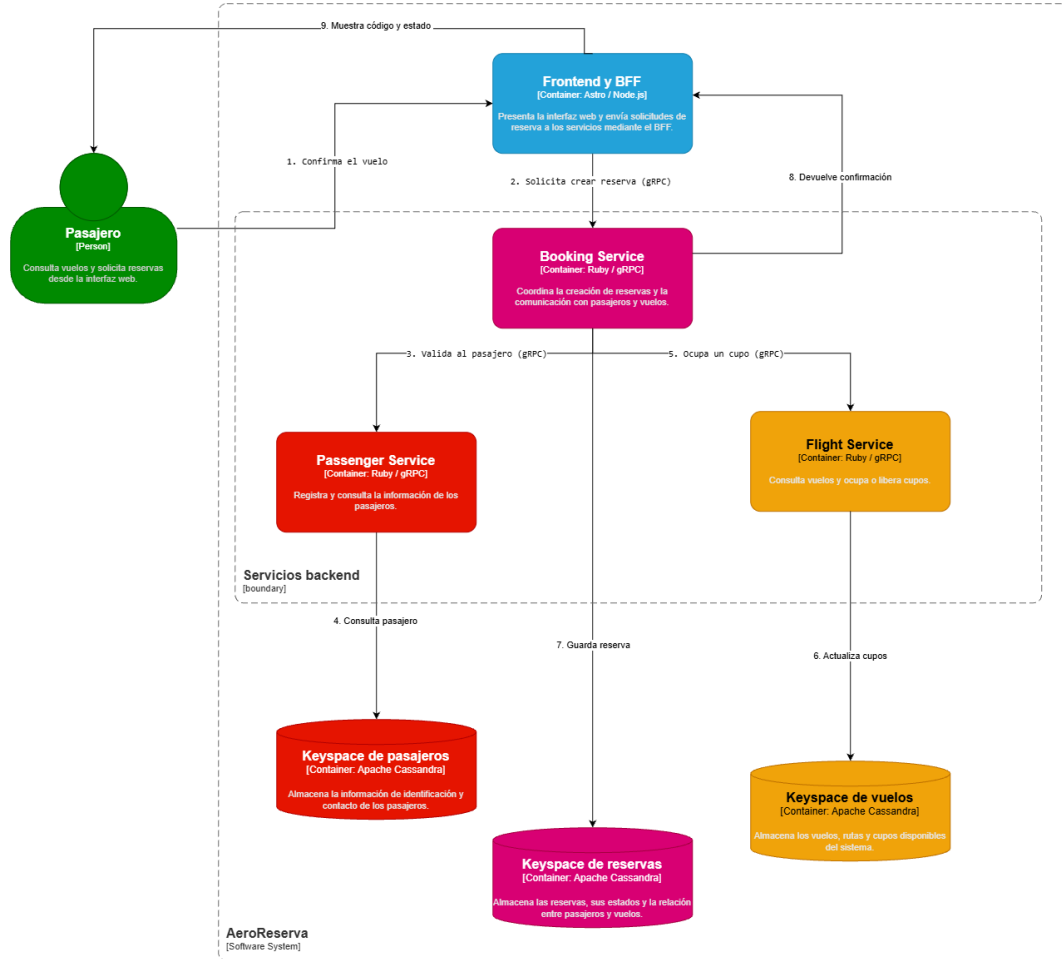


Figura 7: Diagrama C4 dinámico de creación de una reserva

La Figura 7 presenta el orden de las interacciones: el pasajero confirma un vuelo en el frontend; Booking valida al pasajero con Passenger Service, solicita a Flight Service ocupar un cupo y guarda la reserva. Finalmente, el frontend muestra al pasajero el código y el estado de confirmación.

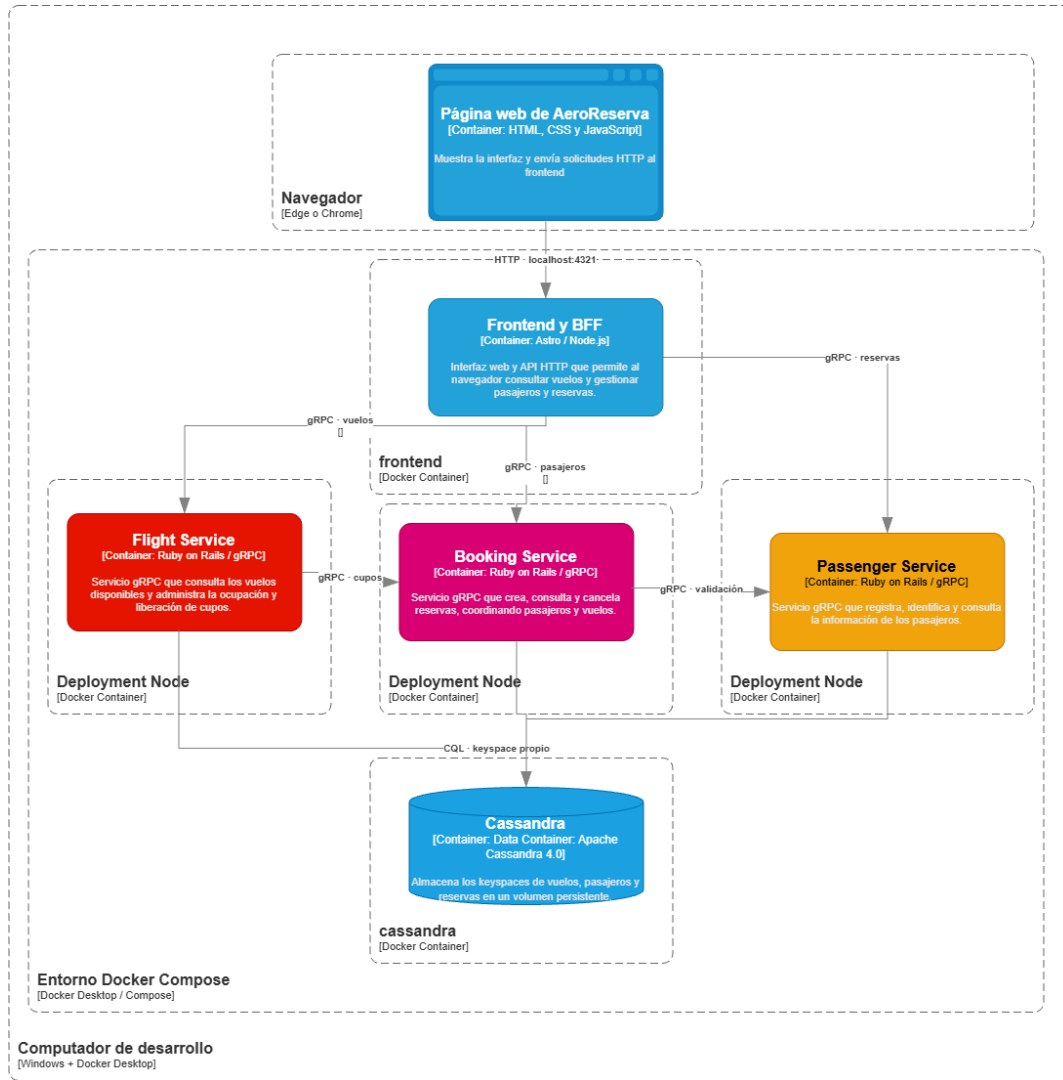


Figura 8: Diagrama C4 de despliegue local de AeroReserva

La Figura 8 muestra la página en el navegador y los contenedores iniciados con Docker Compose: frontend/BFF, tres servicios gRPC y Cassandra. Los servicios almacenan sus datos en *keyspaces* separados dentro de una única instancia de Cassandra con volumen persistente.

4. Backend, gRPC y flujo de negocio

La comunicación entre los tres microservicios se lleva a cabo por un solo archivo de contrato, `proto/aeroreserva.proto`, donde se definen los mensajes y las operaciones disponibles. A partir de ese archivo se genera el código correspondiente a cada lenguaje que se usó, como lo es Ruby en los tres servicios y TypeScript en el frontend, de modo que todos comparten exactamente la misma definición y no existe tanta probabilidad de que se desincronicen. El

contrato agrupa tres servicios:

- **FlightService**: encargado de generar la consulta del listado de vuelos, del detalle de cada uno, de la disponibilidad de cupos, y las operaciones para ocupar o liberar un cupo.
- **PassengerService**: encargado de generar el registro de pasajeros, la consulta por identificador, y también la búsqueda por documento y correo electrónico, sin necesidad de conocer el identificador interno.
- **BookingService**: encargado de generar la creación de reservas, la consulta individual, el listado completo por pasajero y la cancelación.

Cada uno de estos mensajes de vuelo, pasajero, reserva, y sus variantes de solicitud o listado corresponde al paquete de datos que viaja entre servicios. De los tres, Booking Service es el único que depende de los otros dos para completar su función, ya que antes de crear o cancelar una reserva consulta a Passenger Service la existencia del pasajero y a Flight Service la disponibilidad del vuelo. Esta comunicación se encapsula en dos componentes, **FlightClient** y **PassengerClient**. Para esto, ambos incorporan un mecanismo de corte de llamadas en lugar de reintentar sin control ante una falla, que deja de invocar al servicio dependiente cuando este ha fallado de forma reiterada (este mecanismo se describe como Circuit Breaker en la Sección 7 y en el ADR-005 de la Sección 8).

Cuadro 9: Mensaje Booking (el dato central de este servicio)

Campo	Tipo	De dónde sale
id	identificador único	Se genera al crear la reserva, antes de registrar cualquier dato, con el fin de permitir deshacer la operación si ocurre una falla posterior.
booking_code	código legible	Se genera junto con el id, con un formato legible (por ejemplo, "RES-4F2A1B") para facilitar su identificación.
passenger_id	identificador del pasajero	Se resuelve a partir de la sesión iniciada del pasajero; este identificador ya no es proporcionado por el usuario.
flight_id	identificador del vuelo	Se obtiene a partir del número de vuelo seleccionado en la interfaz.
created_at	fecha y hora	Fecha y hora en que se preparó la reserva.
status	estado	Queda en CONFIRMED al crearse, y pasa a CANCELLED si la reserva se cancela o si su creación falló a mitad de camino.

Cuadro 10: Códigos de error comunes a los tres microservicios

Código	Número	Cuándo aparece en <code>booking-service</code>
<code>INVALID_ARGUMENT</code>	3	El identificador de pasajero, vuelo o reserva no tiene un formato válido.
<code>NOT_FOUND</code>	5	El pasajero, vuelo o reserva consultado no existe.
<code>FAILED_PRECONDITION</code>	9	El vuelo ya no tiene cupos, o la reserva ya estaba cancelada.
<code>UNAVAILABLE</code>	14	El otro servicio no respondió a tiempo, o el sistema ha decidido no invocarlo temporalmente.
<code>INTERNAL</code>	13	La reserva no pudo confirmarse debido a un error interno.

4.1. Flujo de creación de reserva

El siguiente es el flujo que se tiene para la creación de una reserva, contemplando en todo momento el uso de identificación por sesión y el manejo de fallos:

1. El pasajero se identifica mediante documento y correo electrónico (sin contraseña) o crea un perfil nuevo. El sistema se encarga de almacenar su identidad en una *cookie* de sesión protegida; a partir de ese momento, el navegador no necesita conocer ni manejar el identificador técnico del pasajero.
2. Desde la pantalla de reserva, el frontend informa al servidor únicamente el número del vuelo seleccionado; el servidor determina internamente la identidad del pasajero (a partir de la sesión) y el vuelo correspondiente a ese número.
3. La solicitud llega a `booking-service`, el cual se encarga de iniciar el proceso de creación de la reserva.
4. Se valida el formato de los identificadores de pasajero y vuelo; si alguno es inválido, la solicitud se rechaza de inmediato.
5. Se confirma la existencia del pasajero mediante una consulta a `passenger-service`.
6. Se confirma la existencia del vuelo y la disponibilidad de cupos mediante una consulta a `flight-service`.
7. Se generan el identificador y el código de la reserva antes de registrar cualquier dato, lo que permite deshacer la operación si ocurre una falla posterior.
8. Se ocupa el cupo en `flight-service`. Si la respuesta se demora y no es posible determinar con certeza si la operación se aplicó, el sistema no la reintenta ni intenta deshacerla, para evitar una corrección basada en información incierta.

9. Si el cupo se ocupa correctamente, la reserva se guarda con estado confirmado.
10. Si el paso anterior falla, el sistema ejecuta una compensación en la que marca la reserva como cancelada (sin eliminar el registro, para conservar evidencia de su existencia) y libera el cupo ocupado. Estas acciones se ejecutan de forma independiente, de modo que el fallo de una no impide la otra.
11. Si el proceso se completa correctamente, se devuelve al pasajero la reserva confirmada junto con su código.

Este flujo se basa en el patrón Saga con compensación, ya que el sistema tiene la capacidad de deshacer sus propios pasos si el proceso falla a mitad de camino. El patrón se encuentra implementado y validado mediante pruebas automatizadas, no solo especificado. La única situación en la que el sistema no ejecuta ninguna corrección es cuando el resultado de ocupar el cupo es ambiguo; por ejemplo, si la respuesta se pierde en el camino, dado que deshacer una operación sin certeza sobre su resultado podría generar un estado peor que no corregir nada.

4.2. Endpoints BFF

El frontend expone hacia el navegador las siguientes rutas, cada una de las cuales está encargada de traducir una acción del usuario (consultar vuelos, identificarse, reservar) en la llamada interna correspondiente:

Cuadro 11: Endpoints expuestos por el BFF

Ruta	Método	Para qué sirve	Qué se espera
GET /api/flights	GET	Listar los vuelos activos con sus cupos (sin exponer ningún identificador interno).	200 si hay vuelos y cupos (503 si flight-service no responde o está siendo evitado temporalmente).
GET /api/passengers	GET	Determinar si existe una sesión abierta y a quién corresponde.	200 si hay sesión, 401 si no.
POST /api/passengers	POST	Iniciar sesión con documento y correo, o crear un perfil nuevo; en ambos casos deja abierta la sesión.	200/201 si se inicia sesión (400 datos inválidos, 404 si no encuentra el perfil, 409 documento duplicado).
DELETE /api/passengers	DELETE	Cerrar la sesión.	200 si se logra cerrar la sesión.
GET /api/bookings	GET	Listar únicamente las reservas de la persona conectada, con los datos de su vuelo.	200 si se logra listar las reservas (401 sin sesión).

Ruta	Método	Para qué sirve	Qué se espera
POST /api/bookings	POST	Crear una reserva para la persona conectada, a partir del vuelo elegido.	201 si se genera la reserva (401 sin sesión, 404 si el vuelo ya no existe, 409/503 según el problema).
DELETE /api/bookings	DELETE	Cancelar una reserva propia, identificándola por su código (se verifica primero que pertenezca a esa persona).	200 si se puede cancelar (401 sin sesión, 404 si el código no le pertenece).

Estas rutas se implementan en un mismo módulo del frontend y comparten tres mecanismos incorporados: una conexión propia hacia cada microservicio, el registro de la identidad del pasajero conectado mediante una *cookie* firmada digitalmente (no falsificable) con una expiración de ocho horas, y una verificación de que la operación se origina en la propia aplicación y no en un sitio externo, como protección básica frente a solicitudes no autorizadas. Cuando un servicio interno reporta un error, esta misma capa lo traduce a un mensaje comprensible y a un código de respuesta HTTP adecuado, en lugar de exponer el error técnico original.

Gracias a esto, ya no es necesario escribir ni mostrar ningún identificador técnico en pantalla: la identidad del pasajero se mantiene en su sesión, y las reservas se identifican siempre por su código legible, no por su identificador interno.

5. Cassandra y modelo de datos

Cassandra se eligió por su capacidad para manejar grandes volúmenes de lecturas y escrituras cuando las consultas que se realizarán sobre los datos se conocen de antemano, a diferencia de una base de datos relacional tradicional, donde las tablas pueden combinarse con mayor libertad (Apache Cassandra, s.f.-f). Este enfoque es coherente con la arquitectura de microservicios, ya que cada servicio diseña sus propias tablas en función de sus necesidades de consulta, sin depender de las tablas de los demás. Como contrapartida, un cambio en la forma de consultar un dato exige crear una tabla nueva, y la duplicación de un mismo dato en varias tablas implica mantenerlas sincronizadas.

5.1. Espacio de datos y tablas de reservas

Cuadro 12: Espacios de datos (*keyspaces*) de AeroReserva

Espacio de datos	Qué guarda	Para qué se usa
aeroreserva_flights	Vuelos y catálogo	Consultar rutas y cupos disponibles.
aeroreserva_passengers	Pasajeros	Registrar personas, identificarlas por documento y correo, y evitar duplicados.
aeroreserva_bookings	Reservas	Crear, consultar, cancelar y corregir reservas.

Todo este esquema de *keyspaces* y tablas se crea y se puebla con datos de ejemplo antes del arranque de los microservicios, tal como se documenta en la Sección 6.3 (Apache Cassandra, s.f.-b).

Las tablas de reservas se diseñaron una por cada forma de consultarlas (Apache Cassandra, s.f.-a, s.f.-d):

```
-- bookings_by_id: para consultar una reserva por su identificador
CREATE TABLE bookings_by_id (id uuid PRIMARY KEY, booking_code text,
    passenger_id uuid, flight_id uuid, created_at timestamp, status text);

-- bookings_by_passenger: para ver el historial de reservas de una persona
CREATE TABLE bookings_by_passenger (passenger_id uuid, created_at timestamp,
    id uuid, booking_code text, flight_id uuid, status text,
    PRIMARY KEY ((passenger_id), created_at, id));

-- bookings_by_status: para listar las reservas segun su estado
CREATE TABLE bookings_by_status (status text, created_at timestamp, id uuid,
    booking_code text, passenger_id uuid, flight_id uuid,
    PRIMARY KEY ((status), created_at, id));
```

La última tabla presenta una particularidad, ya que el estado de la reserva forma parte de la clave que la organiza: no es posible actualizar una fila para cambiar su estado, sino que es necesario eliminar la fila anterior y crear una nueva. Esto es lo que ejecuta el sistema tanto al cancelar una reserva como al corregirla tras una falla, con la diferencia de que el registro no se elimina por completo, sino que se marca como cancelado para conservar evidencia de su existencia, como se mencionó en la Sección 4.1. En las otras dos tablas sí es posible actualizar directamente, porque en ellas el estado no forma parte de la clave. Un mecanismo similar se aplica a los pasajeros, ya que la tabla que los indexa por documento permite verificar rápidamente si un documento ya está registrado, o localizar a un pasajero existente sin recorrer la tabla completa.

El catálogo de vuelos activos. `flight-service` aplica un enfoque similar para el catálogo, debido a que mantiene los datos completos de cada vuelo en una tabla que constituye la fuente autoritativa de los cupos disponibles, y por separado conserva una proyección con los vuelos activos, destinada únicamente a listarlos con rapidez sin recorrer la totalidad de los vuelos existentes. Aunque esta proyección puede actualizarse con cierto retraso, los cupos mostrados se consultan siempre en la tabla principal, por lo que nunca se presenta un cupo inexistente. Adicionalmente, la actualización de la proyección está protegida para evitar que una actualización tardía sobrescriba un dato más reciente.

6. Despliegue y operación

6.1. Dockerfiles

El despliegue del sistema se basa en cinco imágenes Docker: una compartida por los tres microservicios de *backend*, parametrizada mediante el argumento de compilación `SERVICE`, y una propia para el *frontend* (Docker, s.f.).

El Dockerfile de los microservicios (`services/Dockerfile`) parte de la imagen `ruby:4.0`; instala las herramientas de compilación necesarias para las *gems* nativas del proyecto y define `/app` como directorio de trabajo. Copia primero el `Gemfile` y el `Gemfile.lock` del servicio indicado en `SERVICE` e instala las *gems* antes de copiar el resto del código, lo que aprovecha el sistema de caché por capas de Docker, mientras las dependencias no cambien, esa capa no se reconstruye aunque el código del servicio sí lo haga. Luego copia el código completo del servicio y el contrato gRPC compartido (`proto/`), fija las variables de entorno por defecto (`RAILS_ENV`, `CASSANDRA_HOST`, `GRPC_PORT`), expone el puerto 50051 e inicia el servidor gRPC:

```
FROM ruby:4.0
RUN apt-get update -qq && apt-get install -y --no-install-recommends build-essential &&
    rm -rf /var/lib/apt/lists/*
WORKDIR /app
ARG SERVICE
COPY services/${SERVICE}/Gemfile services/${SERVICE}/Gemfile.lock ./
RUN bundle install
COPY services/${SERVICE}/ ./
COPY proto/ /proto/
ENV RAILS_ENV=production
ENV CASSANDRA_HOST=cassandra
ENV GRPC_PORT=50051
EXPOSE 50051
CMD ["bundle", "exec", "ruby", "bin/grpc_server"]
```

El Dockerfile del *frontend* (`frontend/Dockerfile`) sigue la misma lógica sobre Node.js: parte de `node:24-bookworm-slim`, instala las dependencias de `npm` antes de copiar el código fuente, copia el contrato `proto/` compartido, compila el proyecto Astro con `npm run build` y arranca el servidor ya compilado en el puerto 4321:

```
FROM node:24-bookworm-slim
WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
COPY proto/ ../proto/
RUN npm run build
ENV NODE_ENV=production
ENV HOST=0.0.0.0
ENV PORT=4321
EXPOSE 4321
CMD ["node", "./dist/server/entry.mjs"]
```

6.2. docker-compose.yml

El archivo `docker-compose.yml` centraliza la dirección de los seis contenedores que componen la arquitectura: el motor de base de datos Cassandra, un contenedor de inicialización de esquema (`cassandra-init`), los tres microservicios gRPC (`flight-service`, `passenger-service`, `booking-service`) y el *frontend*. Cada servicio depende explícitamente de los anteriores mediante condiciones de salud (`service_healthy`) o de finalización (`service_completed_successfully`), de modo que el sistema arranca en un orden determinista: Cassandra primero, luego `cassandra-init` prepara el esquema y los datos de prueba, después los tres microservicios —`booking-service` espera además a que `flight-service` y `passenger-service` estén saludables, ya que depende de ambos— y por último el *frontend*, que espera a que los tres microservicios estén disponibles.

Cassandra se configura con 512 MB de memoria para la máquina virtual de Java (`MAX_HEAP_SIZE`) y 100 MB para la generación inicial de objetos (`HEAP_NEWSIZE`), y persiste sus datos en el volumen `aeroreserva_cassandra_4_data`. `booking-service`, además, recibe por variables de entorno los parámetros del mecanismo de Circuit Breaker (`GRPC_TIMEOUT_SECONDS`, `GRPC_FAILURE_THRESHOLD`, `GRPC_RECOVERY_SECONDS`, `GRPC_READ_RETRIES`, `GRPC_RETRY_DELAY_SECONDS`), de modo que estos valores pueden ajustarse sin reconstruir las imágenes. Todos los contenedores comparten una red tipo *bridge* (`aeroreserva-network`), que les permite resolverse entre sí por su nombre de servicio en lugar de por una IP asignada dinámicamente.

6.3. cassandra-init

`cassandra-init` es un contenedor auxiliar y temporal cuya única función es preparar la base de datos antes de que los microservicios inicien. Este crea los *keyspaces* y las tablas del sistema, y carga los vuelos de ejemplo, ejecutando en orden los *scripts* `01-schema.cql` y `02-seed-flights.cql` (Apache Cassandra, s.f.-e). Una vez que ambos *scripts* terminan con éxito, el contenedor finaliza (*exited*, código 0) y no vuelve a ejecutarse ni consume recursos adicionales; además, los demás servicios usan precisamente esa finalización exitosa como condición para arrancar.

6.4. Healthchecks

Cada contenedor define su propio mecanismo de comprobación de salud. Cassandra se considera saludable cuando el comando `cqlsh -e 'DESCRIBE KEYSPACES'` responde correctamente; se ejecuta cada 15 segundos, con un tiempo límite de 10 segundos, un margen inicial de 60 segundos para evitar falsos positivos durante el arranque, y un umbral de 12 fallos consecutivos antes de marcar el contenedor como *unhealthy*. Los tres microservicios de Ruby verifican su disponibilidad abriendo un *socket* TCP hacia su puerto gRPC interno, cada 10 segundos, con un tiempo límite de 5 segundos, un margen inicial de 10 segundos y el mismo umbral de 12 fallos consecutivos.

6.5. Ejecución del sistema

El sistema se levanta con Docker Desktop, donde el indicador “Engine running” en la esquina inferior izquierda debe confirmar que el motor está activo. La primera ejecución puede tardar varios minutos, ya que Docker debe descargar la imagen de Cassandra y construir las imágenes de los tres servicios y el *frontend* a partir del código del proyecto. El estado de los contenedores se revisa con:

```
docker compose ps
```

Cassandra y los servicios de la aplicación deben aparecer como *healthy* o *running*, mientras que `cassandra-init` aparece como finalizado (*exited*, código 0), comportamiento esperado dado que su única tarea es preparar el esquema y los datos de prueba. Una vez que los servicios están *healthy*, la aplicación queda disponible en `http://127.0.0.1:4321`. Los registros de todos los servicios pueden consultarse con:

```
docker compose logs -f
```

Y únicamente los de Cassandra con:

```
docker compose logs -f cassandra
```

Finalmente, los contenedores se detienen y se eliminan con:

```
docker compose down
```

6.6. Matriz de ejecución y verificación

La siguiente matriz resume, para cada contenedor, el puerto que expone, de qué depende para arrancar y cómo se comprueba que está listo:

Cuadro 13: Matriz de ejecución y verificación de los contenedores de AeroReserva

Contenedor	Puerto	Depende de	Cómo se verifica
cassandra	9042	(es el primero en arrancar; no depende de ningún otro contenedor)	Healthcheck: <code>cqlsh -e 'DESCRIBE KEYSPACES'</code> .
cassandra-init	(no expone un puerto)	cassandra (<i>healthy</i>)	Finalización exitosa del contenedor (<i>exited</i> , código 0) tras ejecutar los <i>scripts</i> CQL.
flight-service	50051 interno / 50061 en el <i>host</i>	cassandra-init (completado)	Socket TCP abierto en el puerto 50051.
passenger-service	50052	cassandra-init (completado)	Socket TCP abierto en el puerto 50052.
booking-service	50053	cassandra-init (completado); flight-service y passenger-service (<i>healthy</i>)	Socket TCP abierto en el puerto 50053.
frontend	4321	flight-service, passenger-service y booking-service (los tres <i>healthy</i>)	No define <i>healthcheck</i> propio, ya que su arranque queda condicionado a que los tres servicios <i>backend</i> estén <i>healthy</i> .

6.7. Riesgos operativos y de despliegue

Esta matriz recoge los riesgos propios de operar el sistema con Docker Compose:

Cuadro 14: Riesgos operativos y de despliegue de AeroReserva

Riesgo	Cuándo se manifiesta	Mitigación
Arranque prematuro de un microservicio	Si Cassandra tarda más de lo previsto en formar el clúster, especialmente la primera vez que se crea el volumen.	El <i>healthcheck</i> de Cassandra otorga 60 segundos de margen inicial y hasta 12 reintentos; los microservicios solo inician cuando <code>cassandra-init</code> confirma que el esquema quedó cargado.

Riesgo	Cuándo se manifiesta	Mitigación
Primera construcción de imágenes prolongada	Al ejecutar <code>docker compose up -build</code> por primera vez, o tras cambiar una <i>gema</i> o una dependencia de <code>npm</code> .	Construir las imágenes con antelación (<code>docker compose build</code>) antes de la sustentación, en lugar de hacerlo en vivo.
Puertos ocupados en la máquina anfitriona	Si otro proceso ya usa 4321, 9042, 50051–50053 o 50061.	Verificar que los puertos estén libres antes de levantar el sistema, o remapear el puerto en conflicto en <code>docker-compose.yml</code> .
Datos de ejecuciones previas	El volumen <code>aeroreserva_cassandra_4_data</code> persiste entre ejecuciones; una demostración anterior puede dejar cupos ya reservados.	Antes de una demostración, reiniciar el volumen con <code>docker compose down -v</code> y volver a levantar el sistema para recargar el <i>seed</i> original.
Fallo en cascada por dependencia estricta	Si <code>flight-service</code> o <code>passenger-service</code> no llegan a <i>healthy</i> , <code>booking-service</code> no inicia; si cualquiera de los tres <i>backend</i> falla, el <i>frontend</i> tampoco inicia.	Revisar los registros del servicio específico con <code>docker compose logs -f <servicio></code> para diagnosticar la causa antes de reintentar.
Cambios de resiliencia sin efecto inmediato	Ajustar <code>GRPC_FAILURE_THRESHOLD</code> , <code>GRPC_TIMEOUT_SECONDS</code> u otro parámetro del Circuit Breaker no toma efecto en caliente.	Modificar la variable de entorno y recrear únicamente ese contenedor (<code>docker compose up -d -force-recreate booking-service</code>), sin afectar a los demás servicios.

6.8. Guion de demostración

La demostración del sistema sigue el flujo completo de una reserva, de inicio a fin, para evidenciar tanto el funcionamiento normal como los mecanismos de recuperación ante fallas:

1. Levantar el sistema con `docker compose up` y esperar a que todos los contenedores queden en estado *healthy*, verificando con `docker compose ps`.
2. Abrir `http://127.0.0.1:4321` en el navegador y mostrar el listado de vuelos disponibles, usando los filtros de origen y destino.
3. Identificarse con documento y correo electrónico, o registrar un pasajero nuevo, y mostrar que la sesión queda abierta sin exponer ningún identificador técnico en pantalla.
4. Seleccionar un vuelo con cupos disponibles y crear una reserva, mostrando el código de reserva legible que se genera.
5. Consultar el listado de reservas del pasajero y mostrar la reserva recién creada con su estado confirmado.
6. Cancelar la reserva y mostrar que su estado cambia sin que el registro desaparezca del historial.

7. Provocar una falla controlada (por ejemplo, deteniendo temporalmente `flight-service` con `docker compose stop flight-service`) e intentar crear una reserva, mostrando que el sistema responde con el error correspondiente en lugar de quedarse esperando indefinidamente.
8. Reiniciar el servicio detenido (`docker compose start flight-service`) y mostrar que, tras el período de recuperación del Circuit Breaker, el sistema vuelve a aceptar reservas con normalidad.
9. Mostrar en los registros (`docker compose logs -f booking-service`) el momento en que el circuito se abre y se vuelve a cerrar, como evidencia del mecanismo de resiliencia.
10. Cerrar la demostración deteniendo el sistema con `docker compose down`.

7. Calidad, pruebas y resiliencia

7.1. Atributos de calidad evaluados

El sistema se evaluó frente a cinco atributos de calidad relevantes para una arquitectura de microservicios: disponibilidad, rendimiento, consistencia, mantenibilidad y capacidad de recuperación. La Tabla 15 resume, para cada atributo, el mecanismo mediante el cual se logra la evidencia concreta en el código, en la configuración o en las pruebas que permiten verificarlo.

Cuadro 15: Atributos de calidad evaluados en AeroReserva

Cualidad	Cómo se logra	En qué se nota
Disponibilidad	Arranque ordenado y verificado	El sistema completo, la base de datos, los tres servicios y el <i>frontend</i> se despliegan con un solo comando. Cada componente espera a que sus dependencias estén listas antes de iniciar, en lugar de arrancar todos simultáneamente sin verificación.
Rendimiento	Tiempos límite y reintentos controlados	Las llamadas entre servicios cuentan con un tiempo de espera configurable; si un servicio comienza a fallar de forma reiterada, el sistema deja de invocarlo temporalmente en lugar de esperar el tiempo completo en cada intento.
Consistencia	Corrección automática y cupos sin ambigüedad	Si el proceso de creación de una reserva falla a mitad de camino, el sistema deshace lo ejecutado hasta ese punto; además, no es posible que dos solicitudes concurrentes reserven el mismo último cupo, lo cual se verificó mediante una prueba con 12 solicitudes simultáneas sobre 3 cupos disponibles.

Cualidad	Cómo se logra	En qué se nota
Mantenibilidad	Los tres servicios comparten la misma estructura	Al compartir la misma organización interna, cualquier integrante del equipo puede comprender o modificar el servicio de otro sin dificultad.
Recuperación	Un mecanismo de corte por cada servicio del que se depende	Si <code>flight-service</code> o <code>passenger-service</code> comienzan a fallar de forma reiterada, el sistema suspende temporalmente las llamadas hacia ellos y las reanuda de forma gradual e independiente.

7.2. Pruebas de integración

Además de las pruebas unitarias que cubren la lógica interna de cada servicio, el proyecto cuenta con un conjunto de pruebas de integración pensadas específicamente para los puntos donde AeroReserva es más vulnerable: la comunicación entre los tres microservicios por gRPC, la coordinación de pasos de la Saga que crea una reserva, y el acceso concurrente a un mismo cupo en Cassandra. Estas pruebas están agrupadas en archivos como `grpc_resilience_test.rb`, `grpc_transport_test.rb` y `booking_saga_test.rb`; se ejecutan de forma automatizada e independiente de la interacción manual con la interfaz, y reportan de inmediato cualquier falla. Esto permite verificar, antes de una entrega o una demostración, que el sistema realmente se comporta como se espera bajo fallas de red, reintentos y solicitudes simultáneas.

- `grpc_resilience_test.rb`: verifica el correcto funcionamiento del mecanismo de corte de llamadas, que deje de invocar al servicio cuando corresponde, que realice un único intento controlado de recuperación, y que no confunda una respuesta tardía con una respuesta válida.
- `grpc_transport_test.rb`: realiza la misma verificación contra un servicio real, no simulado, para confirmar su funcionamiento en condiciones reales de red.
- `booking_saga_test.rb`: verifica que, ante una falla en la creación de una reserva, el sistema deshaga exactamente los pasos correspondientes.
- `saga_integration_test.rb`: realiza la misma verificación contra la base de datos real, provocando una falla efectiva y comprobando que la corrección se aplique correctamente.
- `atomic_seats_integration_test.rb`: ejecuta 12 solicitudes concurrentes sobre un vuelo con 3 cupos disponibles y confirma que exactamente 3 se completan con éxito, mientras las 9 restantes reciben la notificación correspondiente de falta de disponibilidad.

A estas pruebas se suman dos verificaciones adicionales a nivel de equipo: una que recorre el flujo completo desde el navegador, simulando el comportamiento de un usuario real, y otra que verifica la consistencia de los datos de vuelos entre las tablas en que se almacenan. Los *scripts* manuales previos se conservan como revisión adicional, pero ya no constituyen el único mecanismo de validación del sistema.

7.3. Saga

El patrón Saga con compensación se encuentra implementado, ya que si la creación de una reserva falla luego de ocupar el cupo, el sistema deshace la reserva marcándola como cancelada sin eliminar el registro y libera el cupo ocupado. Ambas correcciones se ejecutan de forma independiente, de modo que la falla de una no impide la ejecución de la otra.

Existe un caso en el que el sistema, de forma deliberada, no ejecuta ninguna corrección: cuando la operación de ocupar el cupo falla de manera ambigua, por ejemplo, debido a una demora, no es posible determinar con certeza si el cupo llegó a ocuparse. En ese escenario, deshacer la operación podría resultar en un error mayor, como liberar un cupo que nunca se ocupó o dejar ocupado uno que sí se aplicó correctamente. Este comportamiento se valida mediante pruebas simuladas y mediante una prueba contra el sistema real.

7.4. Circuit Breaker

El mecanismo de Circuit Breaker se encuentra implementado en cada uno de los dos servicios externos de los que depende **booking-service** (Vuelos y Pasajeros); cada uno cuenta con un mecanismo de corte independiente. Si un servicio falla repetidamente, el sistema suspende las llamadas hacia él durante un período determinado, en lugar de continuar reintentando y prolongar el tiempo de espera del usuario. Transcurrido ese período, se realiza un único intento de recuperación: si resulta exitoso, el servicio vuelve a considerarse disponible; si falla, el corte se reactiva.

Las operaciones de lectura admiten un único reintento antes de considerarse fallidas, mientras que las operaciones que modifican estado (como ocupar un cupo) no se reintentan automáticamente, dado que repetir una operación de resultado incierto conlleva un riesgo. Los tiempos y límites del mecanismo son configurables mediante variables de entorno, sin necesidad de modificar el código.

7.5. Control de cupos

La sobreventa de cupos ya no es posible. Se ejecuta en una única operación condicionada: solo se aplica si el número de cupos coincide con el valor esperado; en caso contrario, la operación se rechaza automáticamente y se reintenta con el valor actualizado, hasta un límite de intentos, únicamente cuando se confirma un conflicto real, nunca ante una demora de red cuyo resultado se desconoce.

Adicionalmente, existe una proyección de vuelos activos destinada a agilizar su consulta, que se actualiza de forma independiente, evitando sobrescribir accidentalmente un dato más reciente en caso de retraso. Una prueba con 12 solicitudes concurrentes sobre 3 cupos disponibles confirmó que exactamente 3 se completan con éxito y las 9 restantes son rechazadas, sin excepciones.

7.6. Patrones de diseño GoF implementados

A diferencia de los patrones arquitectónicos y de resiliencia ya documentados en este proyecto (BFF, Saga, Circuit Breaker), los patrones de diseño GoF (Gamma et al., 1994) operan a nivel de clases dentro del código, no a nivel de servicios o contenedores. Antes de esta revisión, ningún patrón GoF estaba implementado ni nombrado explícitamente en AeroReserva. Se identificaron y documentaron dos, ambos con evidencia verificable en el código, sin modificar ningún diagrama C4 ni ninguna decisión arquitectónica (ADR), porque ambos operan estrictamente dentro de componentes ya documentados (el BFF y Booking Service).

Cuadro 16: Patrones de diseño GoF implementados en AeroReserva

Patrón GoF	Categoría	Dónde vive	Qué resuelve	Evidencia
Factory Method (<i>Simple Factory</i>)	Creacional	<code>frontend/src/lib/grpc-client-factory.ts</code> , función <code>createGrpcClients()</code>	Centraliza en un solo lugar la creación de los tres clientes gRPC (Flight, Passenger, Booking), separando su construcción de su uso en <code>server.ts</code> .	Una función con nombre explícito la encapsula y <code>server.ts</code> solo la invoca. Comportamiento idéntico; solo cambia dónde vive el código.

Patrón GoF	Categoría	Dónde vive	Qué resuelve	Evidencia
Decorator (también interpretable como <i>Proxy</i>)	Estructural	<code>services/booking-service/lib/grpc_resilience.rb</code> , clase <code>GrpcResilience</code>	Añade comportamiento transversal (tiempo de espera, reintento controlado, corte de circuito) a las llamadas gRPC hacia Flight y Passenger, sin cambiar la interfaz que <code>FlightClient/PassengerClient</code> exponen al resto del código.	Documentado explícitamente mediante un comentario en el código de cómo este patrón cumple su función, sin alterar su lógica ni su comportamiento.

8. Decisiones arquitectónicas ADR

ADR-001. Separar Vuelos, Pasajeros y Reservas en microservicios

Estado: Aceptada.

Contexto: el taller exige una arquitectura de microservicios. La reserva de vuelos combina tres capacidades con reglas distintas: catálogo y disponibilidad, identidad del pasajero y ciclo de vida de la reserva. Había que fijar límites que permitieran demostrar colaboración entre servicios sin fragmentar cada operación en un servicio diferente.

Alternativas consideradas: un monolito modular reduciría llamadas de red y simplificaría transacciones, pero no cumpliría el estilo requerido. Un servicio por operación crearía demasiadas dependencias para el alcance del taller. La separación por capacidades de negocio permite asignar reglas y datos a un propietario identificable.

Decisión: implementar Flight Service para vuelos y cupos, Passenger Service para pasajeros y Booking Service para reservas. Cada servicio expone su API gRPC y se ejecuta por separado. Booking coordina operaciones de los otros dos mediante sus contratos, sin leer sus tablas directamente.

Consecuencias: las responsabilidades y los límites quedan explícitos y cada servicio puede probarse por separado. La coordinación añade latencia, fallos parciales y trabajo operativo. El despliegue actual en Compose no demuestra por sí mismo escalamiento ni independencia operativa en producción.

Fuentes: Microsoft (s.f.-c, s.f.-b).

ADR-002. Usar gRPC y Protocol Buffers entre servicios

Estado: Aceptada.

Contexto: gRPC pertenece al *stack* exigido y los tres servicios necesitan acordar operaciones y datos sin compartir implementaciones internas. También es necesario que el contrato siga siendo verificable cuando cambian clientes o servidores.

Alternativas consideradas: REST con JSON sería sencillo de inspeccionar manualmente, pero se apartaría del requisito del curso. Definir mensajes por separado en cada servicio permitiría divergencias. Un archivo de contrato común hace visibles las interfaces de la integración.

Decisión: definir servicios, métodos y mensajes en `proto/aeroreserva.proto` y generar los *stubs* Ruby que implementan y consumen el contrato. El navegador usa HTTP a través del BFF; la comunicación interna entre servicios usa gRPC.

Consecuencias: las firmas y estructuras quedan tipadas y explícitas. Cambiar el `.proto` exige regenerar *stubs* y coordinar la compatibilidad entre consumidores y proveedores. Las llamadas síncronas requieren *deadlines* y tratamiento de indisponibilidad.

Fuentes: gRPC (s.f.-b).

ADR-003. Usar Cassandra con un *keyspace* por servicio

Estado: Aceptada.

Contexto: Cassandra forma parte de las tecnologías exigidas. Los datos de vuelos, pasajeros y reservas requieren propietarios definidos, mientras que el despliegue local usa una sola instancia de Cassandra para mantener una demostración reproducible.

Alternativas consideradas: un único *keyspace* con tablas consultadas directamente por todos los servicios simplificaría algunas lecturas, pero acoplaría los esquemas. Tres instancias físicas darían más aislamiento, con mayor costo y complejidad para el entorno local.

Decisión: mantener un *keyspace* para cada dominio. Cada servicio accede a su propio *keyspace*; para consultar o modificar información ajena, llama a la API del servicio propietario. El entorno inicia el esquema y los datos de demostración mediante *scripts* CQL y conserva los datos en un volumen.

Consecuencias: la propiedad de los datos es lógica, no física. Una caída de la instancia compartida afecta a los tres servicios. La separación por *keyspace* no impone por sí sola permisos de acceso; la restricción también debe respetarse en código y configuración.

Fuentes: Apache Cassandra (s.f.-a); Richardson (s.f.-a).

ADR-004. Usar Astro como frontend y BFF

Estado: Aceptada.

Contexto: la persona usuaria interactúa por navegador mediante HTTP, mientras los servicios de negocio exponen gRPC. El proyecto usa Astro y necesita adaptar solicitudes, errores y respuestas a las pantallas de vuelos, identificación y reservas.

Alternativas consideradas: un *gateway* genérico adicional introduciría otro componente para un solo cliente web. Acceder desde el navegador directamente a los servicios expondría el contrato interno y trasladaría allí la integración. El BFF en el servidor Astro se ajusta al alcance de la demostración.

Decisión: servir las páginas con Astro/Node.js y utilizar sus rutas `frontend/src/pages/api/` como *Backend for Frontend*. Estas rutas reciben HTTP, llaman a clientes gRPC desde el servidor y devuelven datos adecuados para la interfaz. Los detalles internos, como el UUID de vuelo o reserva, no se piden al usuario como entrada manual.

Consecuencias: la transformación de datos y errores se concentra en el BFF y el flujo web resulta más claro. Astro pasa a ser una dependencia de las solicitudes y debe mantenerse compatible con el `.proto`. La identificación demostrativa no equivale a autenticación de producción, ni el BFF incluye automáticamente todas las funciones de un *gateway* empresarial.

Fuentes: Astro (s.f.-c); Microsoft (s.f.-a).

ADR-005. Manejo de fallos distribuidos (Saga y Circuit Breaker)

Estado: Aceptada.

Contexto: `booking-service` depende de dos servicios externos (`flight-service` y `passenger-service`) para completar cada operación de negocio. Un fallo parcial, por ejemplo ocupar el cupo y luego fallar al guardar la reserva, puede dejar el sistema en un estado inconsistente; y si uno de esos servicios cae, cada llamada podría bloquearse indefinidamente sin un mecanismo de corte.

Alternativas consideradas:

- Transacción distribuida (2PC) entre los tres servicios: descartada, ya que ni gRPC ni Cassandra la soportan de forma nativa, y añadiría acoplamiento fuerte entre servicios que deben permanecer independientes.
- Saga coreografiada por eventos (cada servicio publica/escucha eventos): descartada por ahora, ya que el *stack* no incluye un *message broker* (Kafka/RabbitMQ) y agregarlo excede el alcance del taller.
- Saga orquestada (`booking-service` coordina y compensa) con Circuit Breaker sobre los clientes gRPC: elegida.

Decisión: se adoptó el patrón de Saga orquestada con Circuit Breaker, y a diferencia de una versión anterior de este documento, hoy ambas piezas ya están implementadas y no solo diseñadas: la compensación (`compensate_creation + release_seat`) vive en `booking_service_impl.rb`, y el *Circuit Breaker* por dependencia vive en `grpc_resilience.rb`, envolviendo a `FlightClient` y `PassengerClient`. El control de cupos, además, se resolvió con *Lightweight Transactions* de Cassandra, cerrando el riesgo de sobreventa que antes dependía únicamente de la Saga.

Consecuencias: el sistema tolera hoy un fallo tardío en `create_booking` sin dejar cupos huérfanos, y deja de bloquearse llamada por llamada ante una caída prolongada de un servicio (el circuito abre y falla rápido en vez de esperar el *timeout* completo cada vez). El equipo acepta conscientemente una ventana de riesgo más angosta que antes: un fallo ambiguo justo en `occupy_seat` no se compensa, porque no hay forma de saber si el cupo se aplicó del lado del servidor sin arriesgar una doble liberación o una liberación indebida. Los umbrales actuales (3 fallos para abrir el circuito, 10 segundos para volver a intentar, 32 reintentos de CAS) son razonables para el alcance del taller, pero no se han probado bajo carga real; si el circuito abre y cierra con demasiada frecuencia durante la sustentación, ese es el primer parámetro a ajustar (variables de entorno en `docker-compose.yml`, sin tocar código).

Fuentes: Fowler (s.f.); Richardson (s.f.-b).

9. Limitaciones y trabajo futuro

El alcance de AeroReserva es demostrar el estilo arquitectónico de microservicios, no construir un sistema de reservas listo para producción. Esta sección documenta explícitamente los límites conocidos del proyecto y las líneas de trabajo que quedarían pendientes si el sistema se llevara más allá del taller.

9.1. Identificación en lugar de autenticación

La identificación de pasajeros por documento y correo, sin contraseña, se implementó únicamente para poder demostrar el flujo completo sin añadir la complejidad de un sistema de autenticación real. La *cookie* de sesión firmada evita que alguien falsifique su identidad dentro de la demostración, pero no reemplaza mecanismos de producción como contraseñas, verificación de correo, *tokens* OAuth o segundo factor.

9.2. Negocio fuera de alcance

El sistema no incluye pagos, selección de silla, ni *check-in* y equipaje: estas funciones se excluyeron desde el diseño para mantener el proyecto enfocado en demostrar la coordinación entre microservicios, y no en replicar todas las operaciones de una aerolínea real.

9.3. Despliegue limitado a una sola máquina

Todo el sistema se levanta con Docker Compose en un único equipo de desarrollo (ver Figura 8, diagrama de despliegue). No existe balanceo de carga entre varias réplicas de un mismo servicio, ni conmutación automática si el equipo local se cae. El despliegue actual demuestra la separación lógica entre servicios, pero no demuestra por sí mismo escalamiento horizontal ni independencia operativa real en producción, tal como señala la consecuencia ya declarada en el ADR-001.

9.4. Cassandra en una sola instancia física

Aunque cada microservicio tiene su propio *keyspace*, los tres conviven en una única instancia de Cassandra dentro de un solo contenedor. Esta separación es lógica, no física, por ende una caída de esa instancia compartida afecta a los tres servicios a la vez, y el proyecto no demuestra las capacidades reales de tolerancia a fallos de un clúster de Cassandra con varios nodos y un factor de replicación mayor a uno, tal como señala la consecuencia ya declarada en el ADR-003.

9.5. Resiliencia no validada bajo carga real

Los umbrales configurados para el Circuit Breaker (tres fallos consecutivos para abrir el circuito, diez segundos de espera antes de reintentar, hasta 32 reintentos de CAS para el control de cupos) se eligieron como valores razonables para el alcance del taller, pero no se han probado bajo condiciones de carga real o tráfico concurrente sostenido, como reconoce explícitamente la consecuencia declarada en el ADR-005. Un volumen de solicitudes mucho mayor podría exigir recalibrar estos valores.

9.6. Observabilidad básica

El sistema se monitorea revisando los registros de cada contenedor por separado con `docker compose logs`, sin un sistema centralizado de métricas o trazabilidad distribuida

(por ejemplo, Prometheus, Grafana o Jaeger) que permita correlacionar automáticamente una falla con el servicio y la solicitud exacta que la originó.

9.7. Trabajo futuro

Si el proyecto continuara más allá del taller, las líneas de trabajo más directas serían desplegar el sistema en un orquestador como Kubernetes, con varias réplicas por servicio y balanceo de carga real entre ellas; además, migrar Cassandra a un clúster de varios nodos con un factor de replicación mayor a uno, para demostrar tolerancia real a la caída de un nodo; sustituir la identificación de demostración por un mecanismo de autenticación de producción (por ejemplo, OAuth2 o JWT con contraseñas); incorporar las funcionalidades de negocio hoy excluidas (pagos, selección de silla, *check-in* y equipaje); añadir observabilidad centralizada con métricas y trazas distribuidas, y ejecutar pruebas de carga reales que permitan calibrar con datos, y no solo con criterio, los umbrales del Circuit Breaker y de los reintentos de CAS.

10. Análisis arquitectónico

10.1. Matriz de atributos de calidad vs. estilo

Se toma como base la misma clasificación ISO/IEC 25010:2023 ya usada en la Sección 2 (Contexto y alcance) del documento, para mantener consistencia terminológica. Para cada atributo se identifica si el estilo de microservicios lo soporta (lo favorece por diseño) o lo limita (introduce una tensión o un costo), con evidencia concreta de AeroReserva.

Cuadro 17: Matriz de atributos de calidad (ISO/IEC 25010) frente al estilo de microservicios

Atributo de calidad (ISO 25010)	¿El estilo lo soporta o lo limita?	Cómo se manifiesta	Evidencia en AeroReserva
Adecuación funcional	Soporta	Cada servicio implementa completamente su propia capacidad de negocio, sin depender de que otro módulo “complete” su función.	Flight Service resuelve por sí solo todo el ciclo de consulta y control de cupos; no necesita a Booking ni a Passenger para cumplir su función.
Eficiencia de desempeño	Limita (parcialmente)	Las llamadas en memoria de un monolito se convierten en llamadas de red, lo que añade latencia y serialización; se mitiga con un protocolo binario eficiente.	booking-service hace dos llamadas gRPC síncronas (a Flight y a Passenger) por cada reserva, en vez de una sola consulta local; se compensa usando gRPC/Protobuf en lugar de JSON/REST.

Atributo de calidad (ISO 25010)	¿El estilo lo soporta o lo limita?	Cómo se manifiesta	Evidencia en AeroReserva
Compatibilidad	Soporta (con contrato explícito)	Un contrato formal (<code>.proto</code>) obliga a que los servicios acuerden tipos y mensajes antes de integrarse, evitando incompatibilidades silenciosas.	<code>proto/aeroreserva.proto</code> es la única fuente de verdad de los mensajes que intercambian Flight, Passenger y Booking.
Capacidad de interacción (usabilidad)	Neutra / mitiga-da por diseño	El estilo no aporta usabilidad por sí mismo; el riesgo es exponer la complejidad interna (gRPC, errores técnicos) al usuario final.	El BFF en Astro traduce errores gRPC (<code>GRPC::Unavailable</code> , <code>NOT_FOUND</code>) a mensajes entendibles, sin exponer UUID ni códigos internos (RF-07).
Fiabilidad	Limita, pero el estilo ofrece patrones para compensarlo	Al no existir transacciones ACID entre servicios, un fallo a mitad de una operación puede dejar datos inconsistentes si no se maneja explícitamente.	El Saga de <code>booking-service</code> libera el cupo ocupado si falla un paso posterior; el Circuit Breaker evita que un servicio caído bloquee al resto esperando <i>timeouts</i> .
Seguridad	Limita	Cada llamada entre servicios es un punto de red adicional que debe protegerse; en un monolito esa superficie no existe.	El proyecto reconoce explícitamente esta limitación: la identificación es demostrativa, no hay autenticación productiva entre servicios (fuera de alcance, Sección 2).
Mantenibilidad	Soporta	Los límites de servicio permiten modificar o probar un dominio sin tocar los otros, siempre que el contrato no cambie.	Flight, Passenger y Booking tienen sus propias suites de pruebas y se pueden modificar de forma independiente, mientras el <code>.proto</code> se mantenga estable.

El estilo de microservicios no mejora todos los atributos de calidad por igual: a cambio de mantenibilidad y adecuación funcional por dominio, se paga un costo real en desempeño, fiabilidad y seguridad, que en AeroReserva se mitiga (no se elimina) con decisiones puntuales como gRPC binario, Saga con compensación y Circuit Breaker.

10.2. Matriz de principios de diseño vs. estilo

Se identifica cómo el estilo de microservicios ayuda a cumplir (o, en algunos casos, tensiona) cada principio, con un ejemplo concreto del código/arquitectura de AeroReserva.

Cuadro 18: Matriz de principios de diseño frente al estilo de microservicios

Principio	Qué exige	Cómo lo cumple el estilo de microservicios	Evidencia en AeroReserva
SOLID — Responsabilidad única	Un módulo debe tener una sola razón para cambiar.	Cada microservicio se delimita por una única capacidad de negocio (<i>Domain-Driven Design</i>).	Flight Service solo cambia si cambian las reglas de vuelos/cupos; Passenger solo si cambian las reglas de pasajeros.
SOLID — Abierto/Cerrado	Extender comportamiento sin modificar código existente.	Los clientes gRPC envuelven cada llamada en una capa de resiliencia extensible sin tocar la lógica de negocio.	La clase <code>GrpcResilience</code> (Circuit Breaker) se añade como envoltorio a <code>FlightClient/PassengerClient</code> sin modificar la implementación de los servicios remotos.
SOLID — Sustitución de Liskov	Una implementación debe poder reemplazar a otra sin romper el contrato.	El contrato <code>.proto</code> define un tipo estable; cualquier implementación del servicio que lo respete es intercambiable.	Un <code>flight-service</code> reescrito en otro lenguaje seguiría siendo válido para <code>booking-service</code> mientras respete <code>aeroreserva.proto</code> .
SOLID — Segregación de interfaces	No forzar a un cliente a depender de métodos que no usa.	Los servicios gRPC exponen métodos específicos y granulares por caso de uso, no una interfaz genérica.	<code>booking-service</code> solo invoca <code>GetFlight/OccupySeat</code> de Flight Service, nunca los métodos del catálogo completo que usa el <i>frontend</i> .
SOLID — Inversión de dependencias	Depender de abstracciones, no de implementaciones concretas.	Los <i>stubs</i> generados por Protobuf son la abstracción; el cliente gRPC depende del contrato, no de cómo está implementado el servicio remoto.	<code>booking-service</code> nunca conoce el modelo de datos interno de Cassandra de <code>flight-service</code> , solo su contrato gRPC.
KISS (<i>Keep It Simple</i>)	Preferir la solución más simple que funcione.	El estilo empuja a mantener cada servicio pequeño y enfocado, en vez de un módulo monolítico con lógica entrelazada.	Docker Compose (en vez de Kubernetes) para orquestar seis contenedores: la opción más simple que demuestra el estilo sin la complejidad de un clúster real (ADR de despliegue).
DRY (<i>Don't Repeat Yourself</i>)	No duplicar conocimiento/lógica.	Un contrato único generado una vez y compartido evita redefinir mensajes en cada servicio.	Los mismos <i>stubs</i> generados desde <code>aeroreserva.proto</code> se usan tanto en el servidor gRPC de cada servicio como en los clientes que lo consumen.

Principio	Qué exige	Cómo lo cumple el estilo de microservicios	Evidencia en AeroReserva
YAGNI (<i>You Aren't Gonna Need It</i>)	No construir funcionalidad que no se necesita todavía.	El alcance de cada microservicio se limita a lo estrictamente requerido por el taller.	Pagos, selección de silla, <i>check-in</i> y autenticación productiva quedan explícitamente fuera de alcance (Sección 2), en vez de construirse “por si acaso”.
PoLA (Principio de mínima sorpresa)	El sistema debe comportarse como el usuario/desarrollador espera.	El BFF centraliza la traducción de errores para que el comportamiento externo sea siempre predecible, sin importar qué servicio falló internamente.	Cualquier error gRPC (<i>timeout</i> , <i>NOT_FOUND</i> , circuito abierto) llega al usuario como un mensaje de error consistente, nunca como una traza técnica inesperada.
Ley de Demeter (“no hables con extraños”)	Un objeto solo debe comunicarse con sus colaboradores inmediatos.	Cada servicio solo llama a los clientes gRPC de los servicios que necesita directamente, nunca accede a los datos internos de un tercero.	booking-service nunca lee el <i>keyspace</i> de Cassandra de flight-service ; solo le pregunta por gRPC (FlightClient), nunca “atraviesa” esa frontera.
STUPID (anti-principios a evitar)	Evitar acoplamiento rígido, código no probable, duplicación y nombres poco descriptivos.	El estilo obliga a desacoplar por contrato y por base de datos, lo que reduce varios de estos anti-patrones por diseño.	<i>Database per Service</i> evita el acoplamiento típico de una base de datos compartida; cada servicio tiene su propia suite de pruebas independiente (evita <i>Untestability</i>).
Composición sobre herencia	Construir comportamiento combinando objetos, no mediante jerarquías de clases.	La resiliencia (Circuit Breaker) se añade <i>componiendo</i> un objeto envoltorio alrededor del cliente gRPC, no heredando de una clase base común.	FlightClient y PassengerClient usan una instancia de GrpcResilience en vez de heredar de una superclase de “cliente resiliente”; el orquestador de la Saga, a su vez, compone ambos clientes en vez de heredar de ellos.

El estilo de microservicios no garantiza estos principios automáticamente: los favorece estructuralmente (límites de servicio, contratos, independencia de despliegue), pero el equipo tuvo que aplicarlos deliberadamente en el diseño de cada servicio.

10.3. Matriz de tácticas arquitectónicas vs. estilo y stack (justificación de los ADR)

Se usa el catálogo de tácticas arquitectónicas de Bass et al. (2012) para mostrar cómo cada decisión arquitectónica ya documentada en la Sección 8 puede justificarse formalmente como la aplicación de una o más tácticas orientadas a un atributo de calidad concreto.

Cuadro 19: Tácticas arquitectónicas aplicadas en cada ADR de AeroReserva

ADR	Atributo de calidad objetivo	Táctica arquitectónica aplicada	Cómo se justifica en el stack de AeroReserva
ADR-001 — Separar Vuelos, Pasajeros y Reservas en microservicios	Modificabilidad	<i>Aumentar la cohesión y reducir el acoplamiento</i> (dividir el módulo por responsabilidad)	Cada servicio agrupa solo la lógica de su propio dominio (vuelos, pasajeros o reservas), permitiendo modificar uno sin tocar los otros, siempre que el contrato gRPC no cambie.
ADR-002 — Usar gRPC y Protocol Buffers entre servicios	Modificabilidad + Desempeño	<i>Usar un intermediario</i> (el contrato <code>.proto</code> como intermediario estable entre clientes y servidores) + <i>reducir el overhead computacional</i> (serialización binaria en vez de texto)	El <code>.proto</code> desacopla la evolución interna de cada servicio de sus consumidores; Protobuf/HTTP2 reduce el costo de CPU y ancho de banda frente a JSON/REST, mitigando la latencia de red introducida por dividir el sistema.
ADR-003 — Usar Cassandra con un <i>keyspace</i> por servicio	Modificabilidad + Escalabilidad (eficiencia de desempeño)	<i>Dividir el módulo</i> aplicado a los datos (<i>Database per Service</i>)	Al no compartir tablas, un cambio en el modelo de datos de un servicio no puede romper accidentalmente a otro; el modelo distribuido de Cassandra habilita, en teoría, escalar horizontalmente cada <i>keyspace</i> por separado (no explotado en el despliegue local de un solo nodo).
ADR-004 — Usar Astro como <i>frontend</i> y BFF	Modificabilidad + Usabilidad	<i>Usar un intermediario</i> (patrón <i>Backend-for-Frontend</i>)	El BFF concentra en un único punto la adaptación entre el contrato HTTP que espera el navegador y el contrato gRPC que exponen los servicios, evitando que ese conocimiento se disperse por el cliente.

ADR	Atributo de calidad objetivo	Táctica arquitectónica aplicada	Cómo se justifica en el stack de AeroReserva
ADR-005 — Manejo de fallos distribuidos (Saga y Circuit Breaker)	Disponibilidad + Fiabilidad	<i>Circuit Breaker</i> (detectar fallas y evitarlas) + <i>Rollback</i> /transacción compensatoria (deshacer el efecto de una operación fallida) + <i>Reintento controlado</i> (solo en lecturas idempotentes)	El Circuit Breaker de GrpcResilience abre el circuito tras fallas consecutivas y rechaza llamadas sin esperar <i>timeouts</i> ; el Saga de booking-service libera el cupo ocupado si un paso posterior falla, en vez de depender de una transacción distribuida (2PC) descartada explícitamente en el ADR.

Ninguna de las cinco decisiones se tomó “porque sí”: cada una corresponde a una táctica arquitectónica reconocida para resolver la tensión de calidad que ese mismo estilo introduce; por ejemplo, dividir en servicios gana modificabilidad pero pierde desempeño y fiabilidad, y gRPC y el Circuit Breaker son, precisamente, las tácticas que compensan esa pérdida.

10.4. Matriz de mercado laboral vs. estilo y stack

Estado del mercado laboral (Estados Unidos, salario anual) y de la popularidad/demanda para el estilo y cada tecnología del *stack*, con fuentes públicas verificables.

Cuadro 20: Mercado laboral (Estados Unidos) para el estilo de microservicios y cada tecnología del stack de AeroReserva

Estilo/Tecnología	Rol de referencia	Salario promedio anual (USD)	Rango típico (percentiles 25–75)	Popularidad / demanda	Proyección
Microservicios (estilo)	Microservices Architect	US\$128 756	US\$91 000–US\$166 000 (top 10 %: US\$180 000)	Alta: sigue siendo el estilo de referencia para sistemas <i>backend</i> distribuidos en la nube; términos como “microservices” aparecen de forma constante en encuestas de arquitectura de <i>software</i> .	Estable/creciente: la adopción de contenedores y arquitecturas <i>cloud-native</i> mantiene la demanda de este perfil por encima del <i>backend</i> generalista.

Estilo/Tecnología	Rol de referencia	Salario promedio anual (USD)	Rango típico (percentiles 25–75)	Popularidad / demanda	Proyección
gRPC + Ruby on Rails (<i>backend</i> interno)	Backend Engineer	US\$178 011 (mediana <i>total pay</i>)	Base: US\$99 000–US\$161 000	Media-alta para el rol de <i>backend</i> en general; gRPC es un requisito frecuente en ofertas de “ <i>backend</i> /microservices engineer” más que un título de puesto propio.	Creciente: gRPC gana terreno frente a REST para comunicación interna por su eficiencia en arquitecturas distribuidas.
Ruby on Rails (específico)	Ruby on Rails Developer	US\$106 459 (<i>total pay</i>)	US\$80 000–US\$149 000	Media, con tendencia a la baja en popularidad general del lenguaje: Ruby salió del <i>top 20</i> del índice TIOBE en 2025 (posición 22, 0.73 %), pero Ruby figura entre las tecnologías mejor pagadas según Stack Overflow por su nicho de desarrolladores <i>senior</i> .	Nicho estable: menos volumen de vacantes que otros lenguajes, pero sueldos competitivos por escasez relativa de talento.
Apache Cassandra	Cassandra Developer / Software Architect con Cassandra	US\$102 399	US\$110 500–US\$119 500 (top: ~US\$145 000)	Media: demanda especializada en bases de datos distribuidas de gran escala (no es la opción por defecto salvo que el volumen de datos lo justifique).	Estable: ligada a proyectos con requisitos reales de escritura masiva distribuida (<i>streaming</i> , IoT, mensajería), más que a proyectos de tamaño pequeño/mediano.

Estilo/Tecnología	Rol de referencia	Salario promedio anual (USD)	Rango típico (percentiles 25–75)	Popularidad / demanda	Proyección
Docker Compose / contenedores	Docker Engineer	US\$101 752	US\$84 000–US\$116 500 (top: US\$135 000)	Muy alta: Docker es una habilidad transversal esperada en casi cualquier vacante <i>backend</i> o DevOps actual.	Creciente: la contenerización sigue siendo estándar de facto para despliegue reproducible, aunque Docker Compose específicamente es más común en desarrollo/demo que en producción a gran escala (donde domina Kubernetes).
Astro (<i>frontend</i> /BFF)	Frontend / Jamstack Developer	Rango observado: US\$102 000–US\$182 000	Varía fuerte por <i>seniority</i> (de <i>front-end junior</i> a <i>principal engineer</i>)	Creciente rápidamente: en la encuesta State of JS 2025, Astro lidera en satisfacción entre <i>meta-frameworks</i> , con una brecha del 39 % sobre Next.js, y pasó de ser prácticamente desconocido a “serio competidor” en pocos años.	Al alza: Astro específicamente todavía no es un requisito masivo de vacantes (es más nuevo que Next.js/Nuxt), pero su curva de adopción y satisfacción sugiere una demanda creciente a mediano plazo.

El estilo de microservicios como tal está consolidado y bien remunerado en el mercado. Dentro del *stack* específico de AeroReserva conviven una tecnología de comunicación en fuerte crecimiento (gRPC), dos tecnologías maduras y bien pagadas pero de adopción más nicho (Ruby on Rails, Cassandra), una práctica transversal casi universal (contenedores/Docker) y un *framework* de *frontend* joven con una curva de satisfacción y adopción muy pronunciada (Astro); es decir, el *stack* combina piezas “seguras” con una apuesta relativamente nueva.

11. Referencias y anexos

Las referencias se presentan en orden alfabético y con sangría francesa, conforme al formato APA (7.^a edición). Los sitios oficiales de documentación sin fecha de publicación identificable se citan como “s.f.”, y cuando un mismo autor u organización tiene más de una obra sin fecha, se distinguen con sufijos de letra (*a*, *b*, *c*...) asignados en orden alfabético por título, como exige la norma APA.

Apache Cassandra. (s.f.-a). *Cassandra Query Language (CQL)*. <https://cassandra.apache.org/doc/4.0/cassandra/cql/>

Apache Cassandra. (s.f.-b). *Cassandra quickstart*. https://cassandra.apache.org/doc/4.0/cassandra/getting_started/quickstart.html

Apache Cassandra. (s.f.-c). *Documentation*. <https://cassandra.apache.org/doc/>

Apache Cassandra. (s.f.-d). *Introduction to data modeling*. https://cassandra.apache.org/doc/4.0/cassandra/data_modeling/intro.html

Apache Cassandra. (s.f.-e). *Apache Cassandra* [página oficial]. https://cassandra.apache.org/_/index.html

Apache Cassandra. (s.f.-f). *RDBMS design*. https://cassandra.apache.org/doc/4.0/cassandra/data_modeling/data_modeling_rdbms.html

Astro. (s.f.-a). *Astro documentation*. <https://docs.astro.build/>

Astro. (s.f.-b). *@astrojs/node*. <https://docs.astro.build/en/guides/integrations-guide/node/>

Astro. (s.f.-c). *Endpoints*. <https://docs.astro.build/en/guides/endpoints/>

Astro. (s.f.-d). *On-demand rendering*. <https://docs.astro.build/en/guides/on-demand-rendering/>

Astro. (s.f.-e). *Release content faster: How Firebase cut blog publishing from hours to minutes with Astro* [caso de estudio]. <https://astro.build/case-studies/firebase/>

Astro. (s.f.-f). *Web pages in half the time: Why Microsoft chose Astro to build their Fluent 2 design system website* [caso de estudio]. <https://astro.build/case-studies/microsoft/>

Astro. (s.f.-g). *Why Astro?* <https://docs.astro.build/en/concepts/why-astro/>

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3.^a ed.). Addison-Wesley.

Docker. (s.f.). *Docker documentation*. <https://docs.docker.com/>

Fowler, M. (s.f.). *CircuitBreaker*. <https://martinfowler.com/bliki/CircuitBreaker.html>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

gRPC. (s.f.-a). *Basics tutorial: Ruby*. <https://grpc.io/docs/languages/ruby/basics/>

gRPC. (s.f.-b). *Core concepts, architecture and lifecycle*. <https://grpc.io/docs/what-is-grpc/core-concepts/>

gRPC. (s.f.-c). *gRPC documentation*. <https://grpc.io/docs>

Instaclustr. (s.f.). *Exploring the key features of Cassandra 5.0* [entrada de blog]. <https://www.instaclustr.com/blog/exploring-the-key-features-of-cassandra-5-0/>

Microsoft. (s.f.-a). *Backends for frontends pattern*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends>

Microsoft. (s.f.-b). *Data sovereignty per microservice*. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/data-sovereignty-per-microservice>

Microsoft. (s.f.-c). *Identify microservice boundaries*. <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/microservice-boundaries>

Richardson, C. (s.f.-a). *Pattern: Database per service*. <https://microservices.io/patterns/data/database-per-service.html>

Richardson, C. (s.f.-b). *Pattern: Saga*. <https://microservices.io/patterns/data/saga.html>

Ruby. (s.f.). *About Ruby*. <https://www.ruby-lang.org/en/about/>

Ruby on Rails. (s.f.-a). *Getting started with Rails*. https://guides.rubyonrails.org/getting_started.html

Ruby on Rails. (s.f.-b). *Ruby on Rails guides*. <https://guides.rubyonrails.org/>

Shopify Engineering. (s.f.). *Shopify Engineering blog*. <https://shopify.engineering/>

State of JS. (2025). *State of JS 2025: Front-end frameworks*. <https://stateofjs.com/>